Import necessary libraries

In [12]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.express as px
from pandas.plotting import parallel_coordinates
%matplotlib inline
import warnings
warnings.filterwarnings("ignore")
from sklearn import set_config
set_config(print_changed_only = False)
```

load data set

In [13]:
```
smoking = pd.read_csv("your_data.csv")
smoking = smoking.drop('Unnamed: 0', axis=1)
df = smoking.drop("id", axis = 1)
df.head(n = 15).style.background_gradient(cmap = "Blues")
```

Out[13]:

	hearing(right)	triglyceride	hearing(left)	weight(kg)	eyesight(left)	eyesight(right)	AST	waist(cm)	Cholesterol	hemoglobin	Gtp
0	1	300	1	60	0.500000	0.600000	22	81.000000	172	16.500000	27
1	2	55	2	65	0.600000	0.700000	27	89.000000	194	16.200000	37
2	1	197	1	75	0.400000	0.500000	27	81.000000	178	17.400000	53
3	1	203	1	95	1.500000	1.200000	20	105.000000	180	15.900000	30
4	1	87	1	60	1.500000	1.000000	19	80.500000	155	15.400000	17
5	1	343	1	55	1.200000	1.200000	24	51.000000	199	15.900000	119
6	1	153	1	55	1.500000	1.200000	17	69.000000	222	13.000000	16
7	1	165	1	60	0.700000	0.900000	16	84.500000	282	14.500000	16
8	1	163	1	70	0.700000	1.000000	24	89.000000	243	15.700000	31
9	1	27	1	50	1.500000	1.500000	22	73.000000	183	13.200000	14
10	1	76	1	50	0.800000	0.600000	17	68.000000	165	12.900000	8
11	1	50	1	45	1.200000	1.500000	25	64.000000	217	13.600000	15
12	1	94	1	55	0.800000	0.800000	21	80.000000	163	12.600000	9
13	1	84	1	65	1.200000	1.000000	23	81.000000	215	15.900000	36
14	1	145	1	90	0.800000	0.500000	27	103.000000	220	16.600000	26

drop id & unnammed Column as it is redundant

check Data info

In [14]:
```
df.info()
print("\nnumber of null values: ",df.isnull().sum().sum()," ")
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 159256 entries, 0 to 159255
Data columns (total 12 columns):
 #   Column           Non-Null Count   Dtype  
---  ------           --------------   -----  
 0   hearing(right)   159256 non-null  int64  
 1   triglyceride     159256 non-null  int64  
 2   hearing(left)    159256 non-null  int64  
 3   weight(kg)       159256 non-null  int64  
 4   eyesight(left)   159256 non-null  float64
 5   eyesight(right)  159256 non-null  float64
 6   AST              159256 non-null  int64  
 7   waist(cm)        159256 non-null  float64
 8   Cholesterol      159256 non-null  int64  
 9   hemoglobin       159256 non-null  float64
 10  Gtp              159256 non-null  int64  
 11  smoking          159256 non-null  int64  
dtypes: float64(4), int64(8)
memory usage: 14.6 MB

number of null values:  0
```

No null values exist in the data

Univariate

Descriptive Statistics of Numeric Variables

In [15]:
```
df.describe().T.style.background_gradient(cmap = "Blues")
```

Out[15]:

	count	mean	std	min	25%	50%	75%	max
hearing(right)	159256.000000	1.023421	0.151238	1.000000	1.000000	1.000000	1.000000	2.000000
triglyceride	159256.000000	127.616046	66.188989	8.000000	77.000000	115.000000	165.000000	766.000000
hearing(left)	159256.000000	1.023974	0.152969	1.000000	1.000000	1.000000	1.000000	2.000000
weight(kg)	159256.000000	67.143662	12.586198	30.000000	60.000000	65.000000	75.000000	130.000000
eyesight(left)	159256.000000	1.005798	0.402113	0.100000	0.800000	1.000000	1.200000	9.900000
eyesight(right)	159256.000000	1.000989	0.392299	0.100000	0.800000	1.000000	1.200000	9.900000
AST	159256.000000	25.516853	9.464882	6.000000	20.000000	24.000000	29.000000	778.000000
waist(cm)	159256.000000	83.001990	8.957937	51.000000	77.000000	83.000000	89.000000	127.000000
Cholesterol	159256.000000	195.796165	28.396959	77.000000	175.000000	196.000000	217.000000	393.000000
hemoglobin	159256.000000	14.796965	1.431213	4.900000	13.800000	15.000000	15.800000	21.000000
Gtp	159256.000000	36.216004	31.204643	2.000000	18.000000	27.000000	44.000000	999.000000
smoking	159256.000000	0.437365	0.496063	0.000000	0.000000	0.000000	1.000000	1.000000

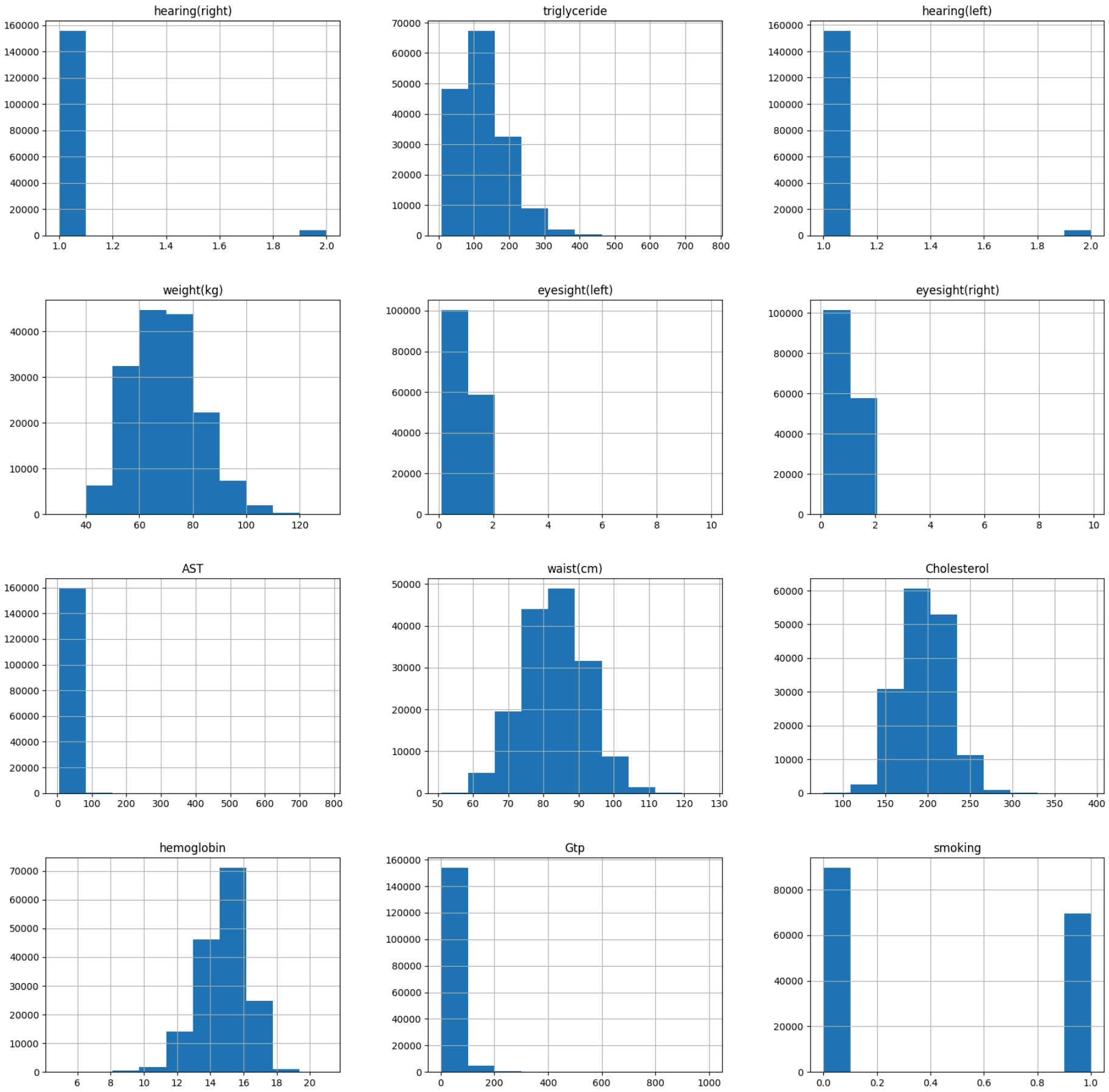
In [16]:
```
features = df.drop("smoking",axis=1)
for f in features:
    unique_values = df[f].nunique()
    print(f"Number of Unique values in the {f} column:", unique_values)
```

```
Number of Unique values in the hearing(right) column: 2
Number of Unique values in the triglyceride column: 392
Number of Unique values in the hearing(left) column: 2
Number of Unique values in the weight(kg) column: 28
Number of Unique values in the eyesight(left) column: 20
Number of Unique values in the eyesight(right) column: 17
Number of Unique values in the AST column: 140
Number of Unique values in the waist(cm) column: 531
Number of Unique values in the Cholesterol column: 227
Number of Unique values in the hemoglobin column: 134
Number of Unique values in the Gtp column: 362
```

hearing(right) & hearing(left) are categorical features

Histogram

In [17]:
```
df.hist(figsize = (20, 20));
```



From the histogram, Some features are skewed and would be preprocessed during feature engineering phase

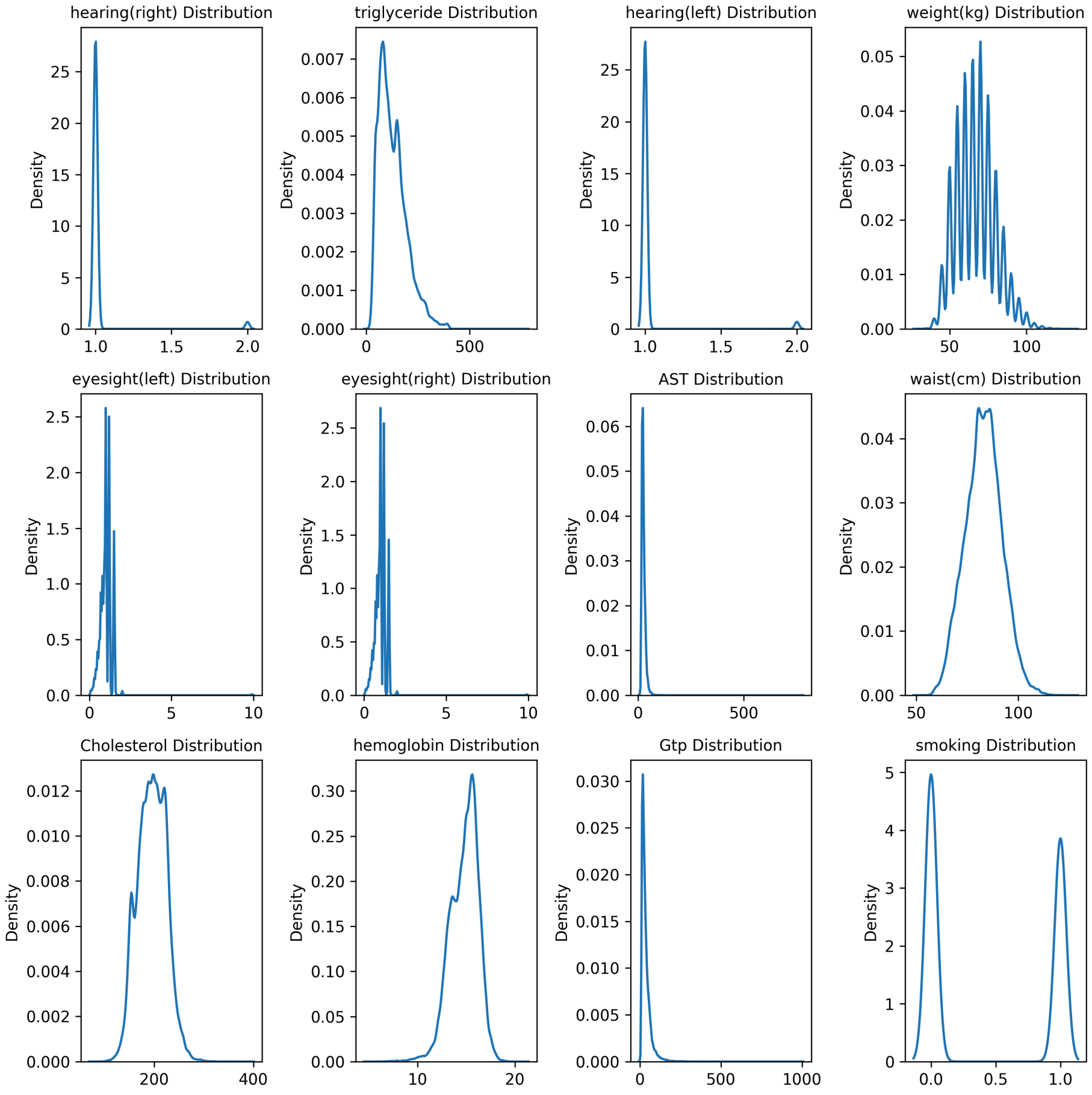
KDE plot

In [29]:
```
fig, ax = plt.subplots(3, 4, figsize = (10, 10), dpi = 300)
ax = ax.flatten()

for i,column in enumerate(df):

    sns.kdeplot(df[column], ax=ax[i])

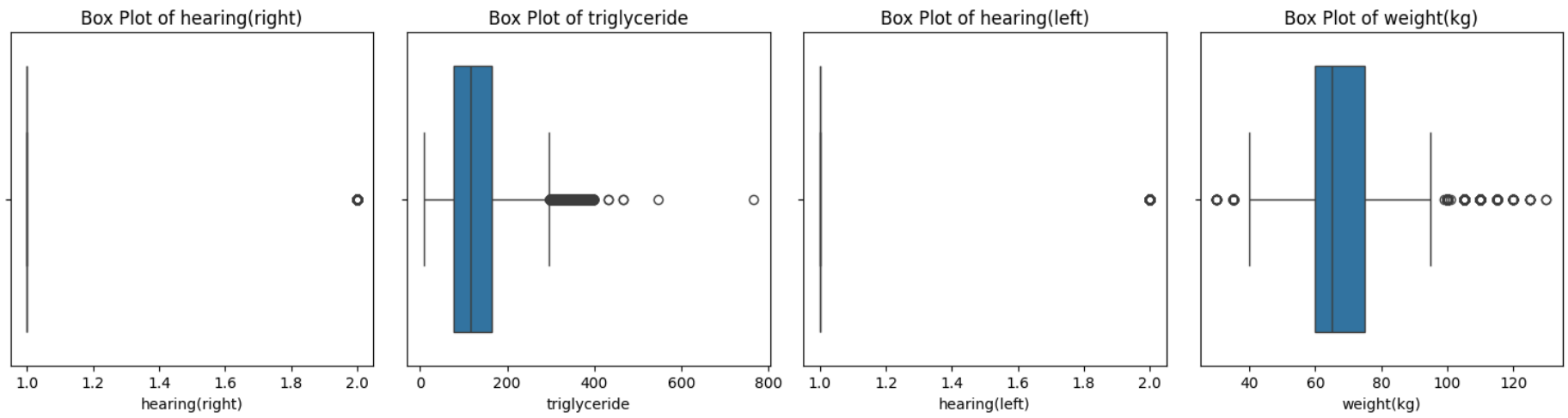
    ax[i].set_title(f'{column} Distribution', size = 10)
    ax[i].set_xlabel(None)
plt.tight_layout()
```

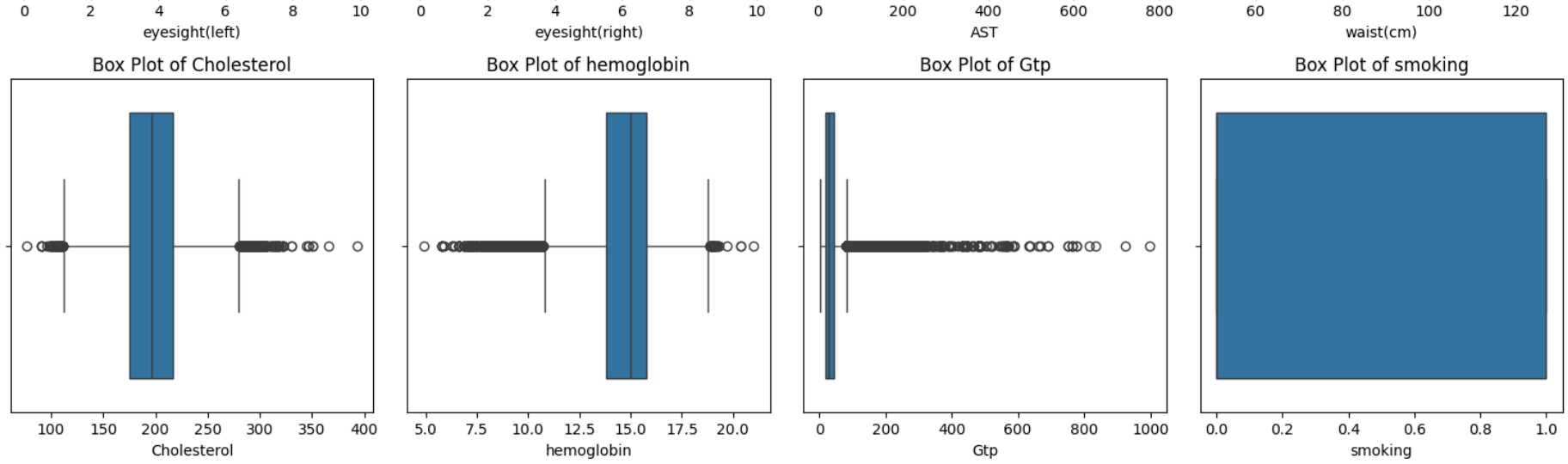


Kernel Density Estimate Plot shows More details about skewing that was spotted in the histogram plot

Box Plot

In [18]:
```
fig, ax1 = plt.subplots(3, 4, figsize=(15, 12))
plt.subplots_adjust(bottom=0.1, right=1, top=0.9, hspace=0.4)
for i in range(len(df.columns)):
    sns.boxplot(data=df, x=df.columns[i], ax=ax1[i//4, i%4])
    ax1[i//4, i%4].set_title(f"Box Plot of {df.columns[i]}")
plt.tight_layout()
plt.show()
```





box plot of each feature shows percentiles and outliers

Bar plot Plot

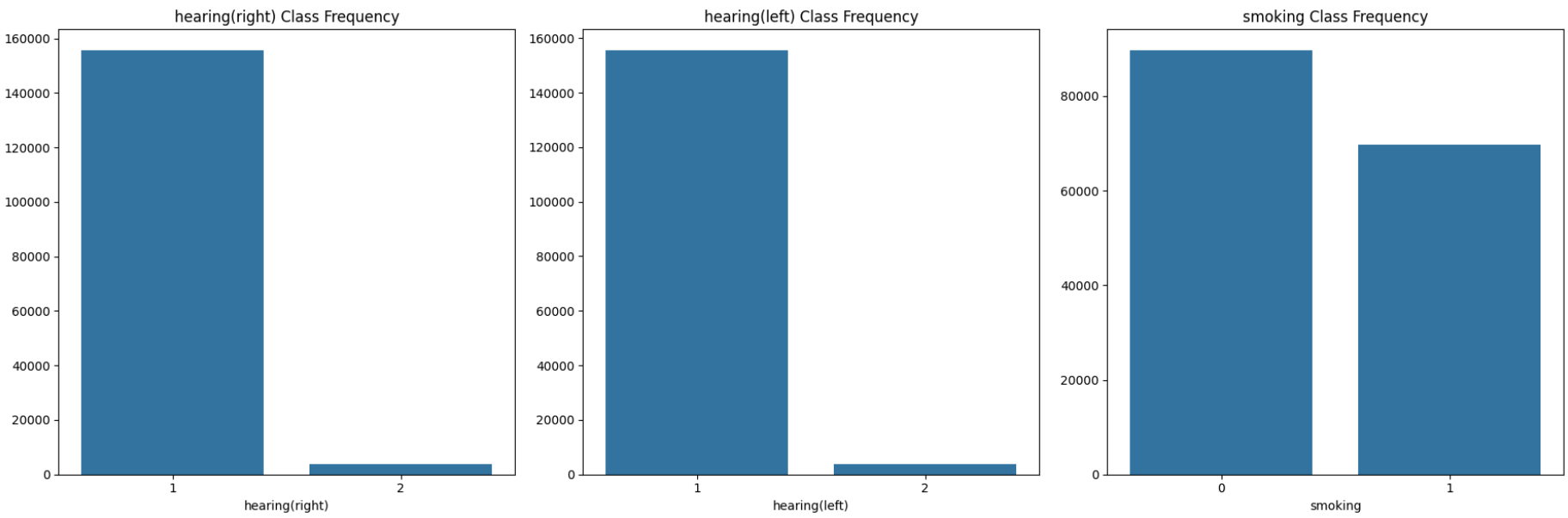
In [19]:
```
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
axes = axes.flatten()

sns.barplot(ax=axes[0], x=df["hearing(right)"].value_counts().index,
            y=df["hearing(right)"].value_counts().values).set(title="hearing(right) Class Frequency")

sns.barplot(ax=axes[1], x=df["hearing(left)"].value_counts().index,
            y=df["hearing(left)"].value_counts().values).set(title="hearing(left) Class Frequency")

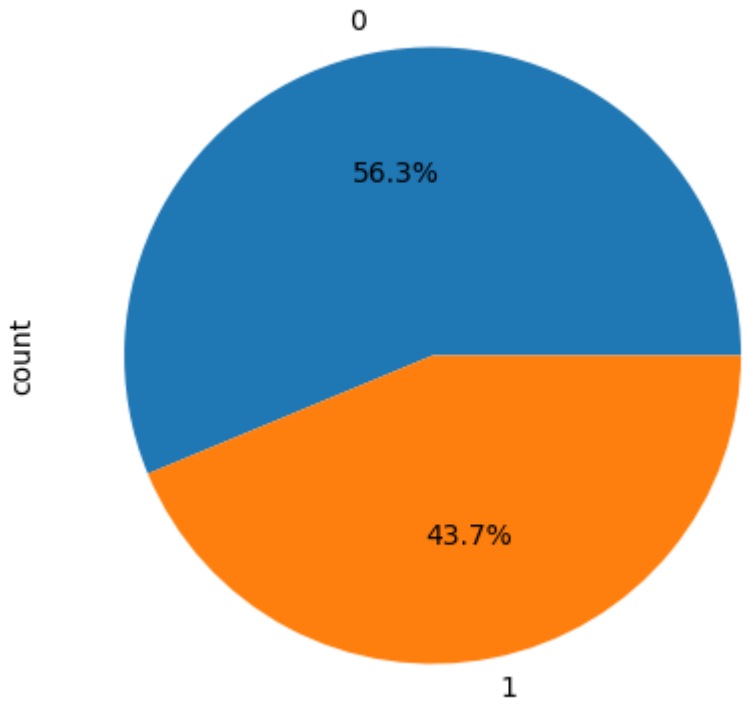
sns.barplot(ax=axes[2], x=df["smoking"].value_counts().index,
            y=df["smoking"].value_counts().values).set(title="smoking Class Frequency")

plt.tight_layout()
plt.show()
```



target pie plot

In [20]:
```
plt.figure(figsize = [5, 5])
df["smoking"].value_counts().plot.pie(autopct='%1.1f%%');
```

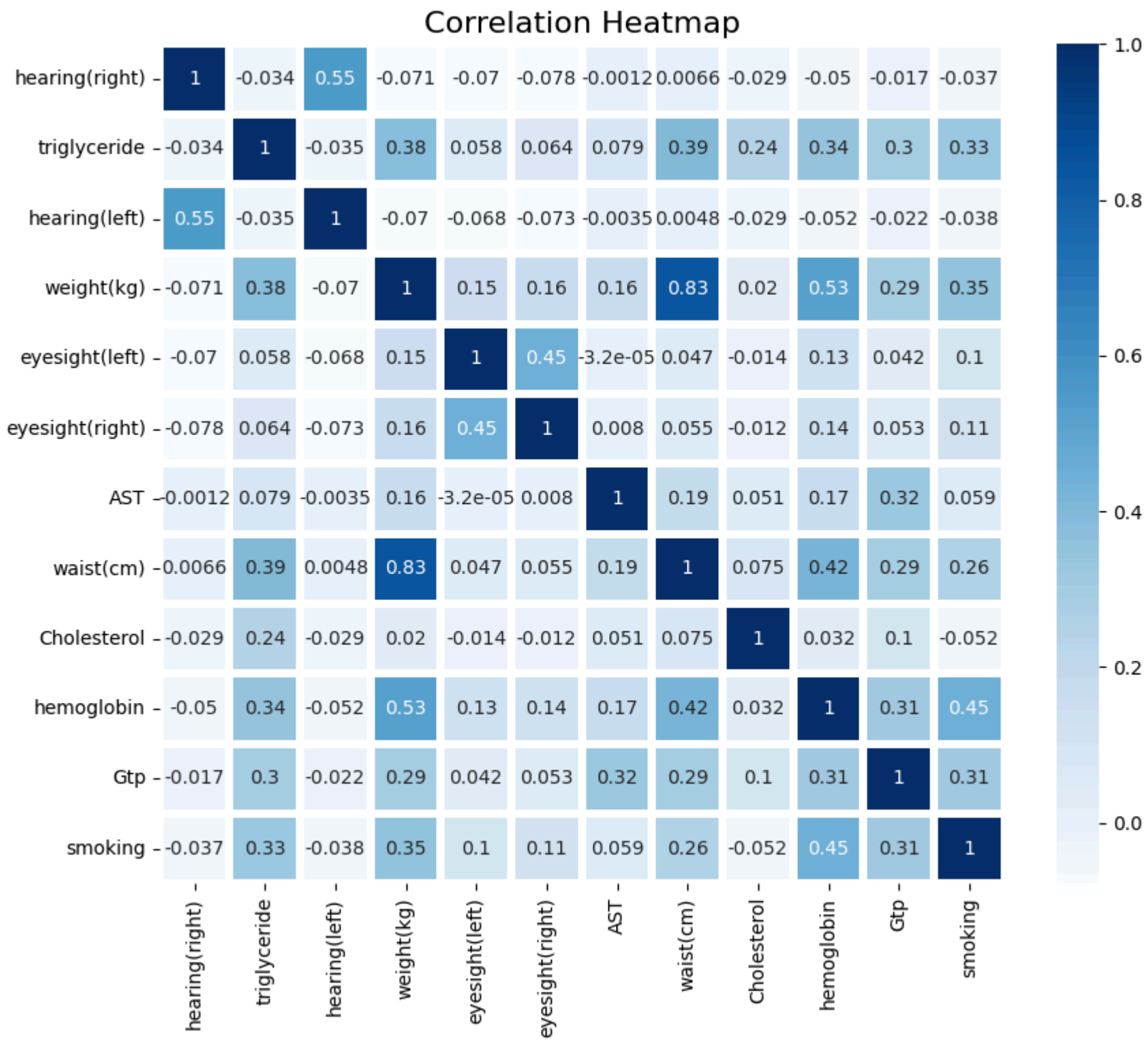


Visualization of non categorical Features and target

Bivariate

Correlation and Heatmap

In [21]:
```
plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(), annot=True, square=False, linewidths=3,
            linecolor="white", cmap="Blues")
plt.title("Correlation Heatmap", fontsize=16)
plt.show()
```



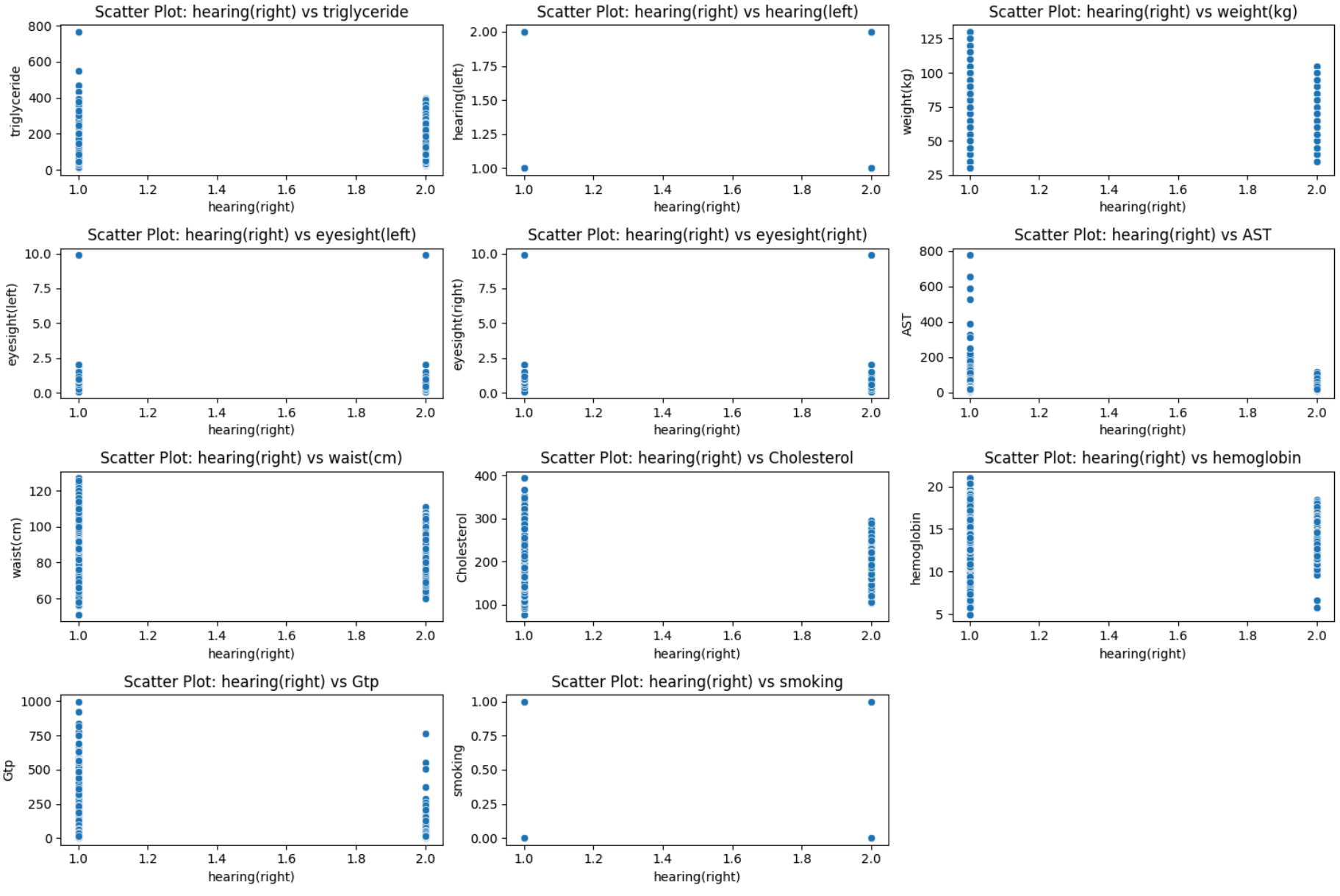
strongly correlated Features: (weight-waist),(hearingL-hearingr),(hemoglobin-weight)

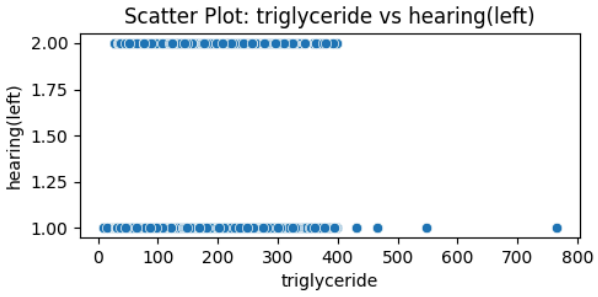
top 3 correlated Features with target: (hemoglobin,weight,triglyceride)

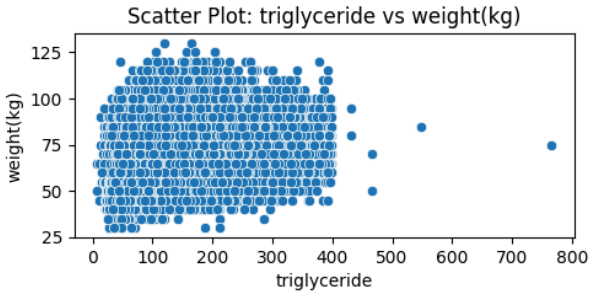
Scatter plot

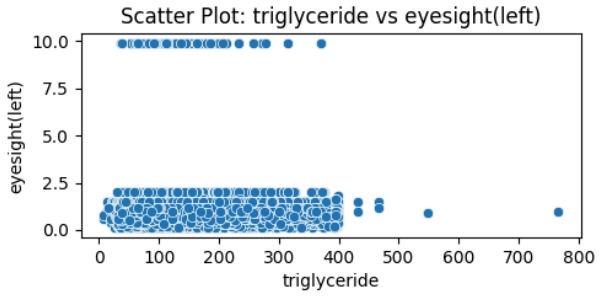
In [22]:
```
numerical_columns = df.select_dtypes(include=['float64', 'int64']).columns

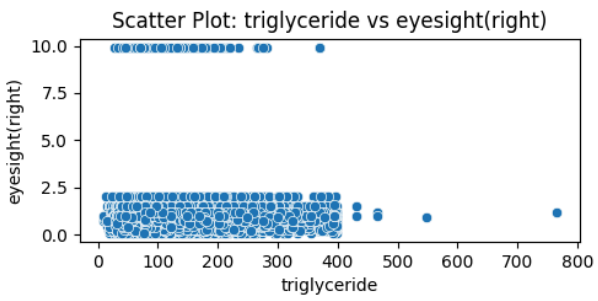
for i in range(len(numerical_columns)-1):
    plt.figure(figsize=(15, 10))
    num_plots = len(numerical_columns) - (i + 1)
    rows = (num_plots // 3) + 1
    for j in range(i + 1, len(numerical_columns)):
        plt.subplot(rows, 3, j - i)
        sns.scatterplot(x=df[numerical_columns[i]], y=df[numerical_columns[j]])
        plt.title(f'Scatter Plot: {numerical_columns[i]} vs {numerical_columns[j]}')
        plt.xlabel(numerical_columns[i])
        plt.ylabel(numerical_columns[j])
    plt.tight_layout()
    plt.show()
```

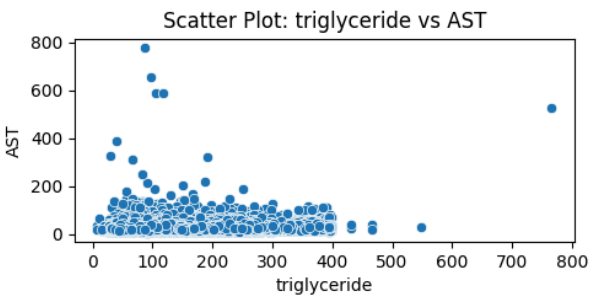


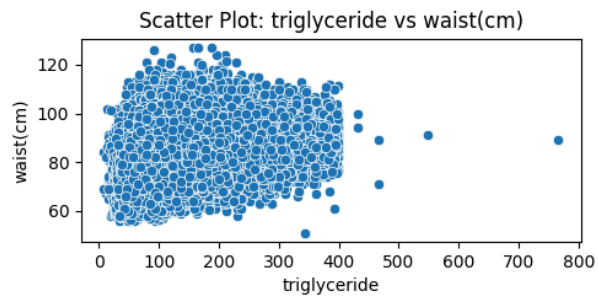

Scatter Plot: triglyceride vs hearing(left)
hearing(left)
triglyceride

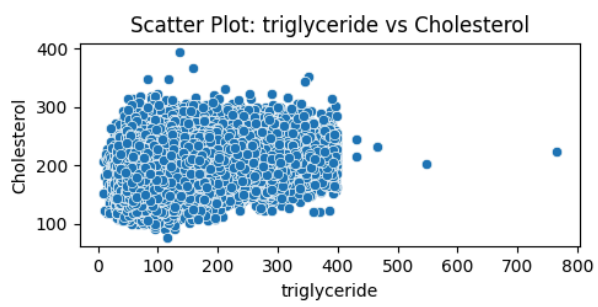

Scatter Plot: triglyceride vs weight(kg)
weight(kg)
triglyceride

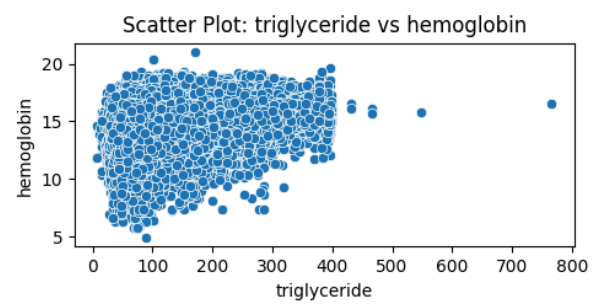

Scatter Plot: triglyceride vs eyesight(left)
eyesight(left)
triglyceride

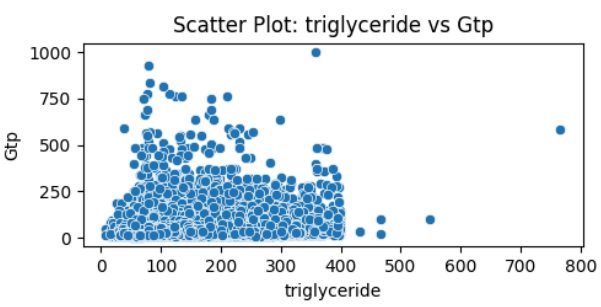

Scatter Plot: triglyceride vs eyesight(right)
eyesight(right)
triglyceride

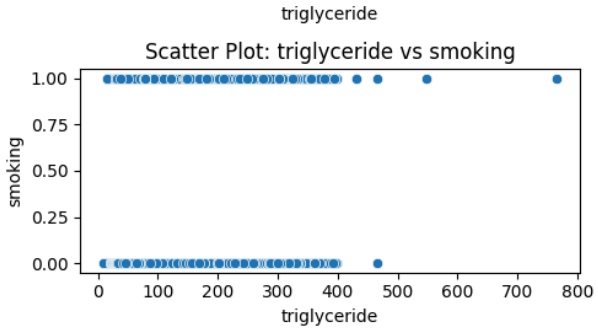

Scatter Plot: triglyceride vs AST
AST
triglyceride

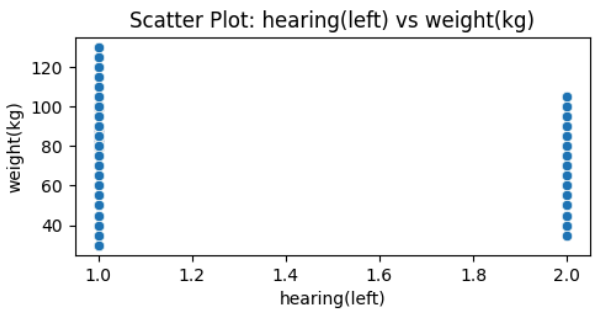

Scatter Plot: triglyceride vs waist(cm)
waist(cm)
triglyceride

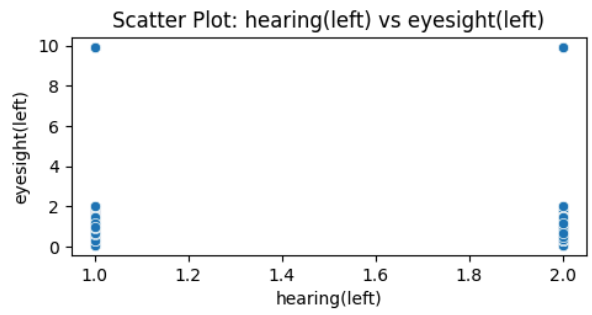

Scatter Plot: triglyceride vs Cholesterol
Cholesterol
triglyceride

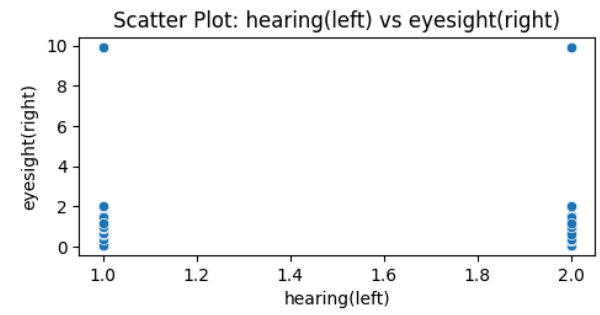

Scatter Plot: triglyceride vs hemoglobin
hemoglobin
triglyceride

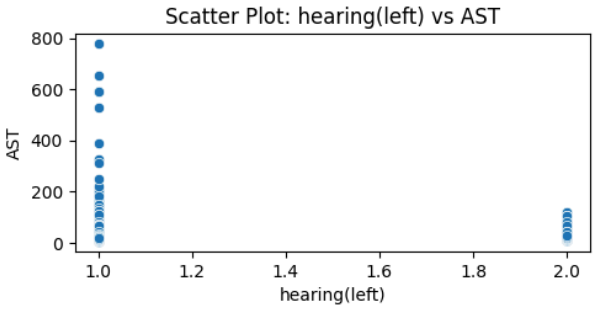

Scatter Plot: triglyceride vs Gtp
Gtp
triglyceride

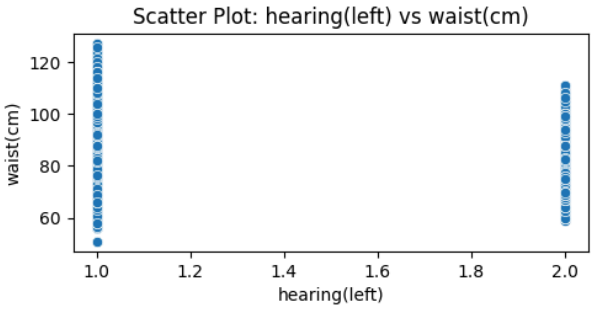

Scatter Plot: triglyceride vs smoking
smoking
triglyceride

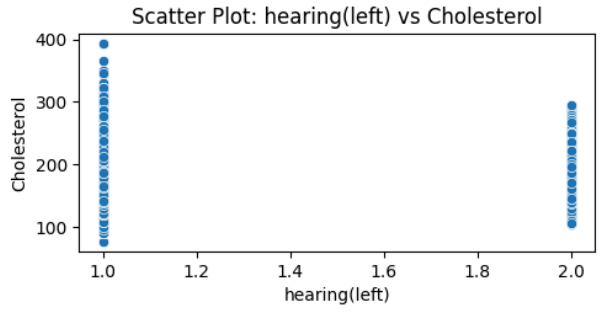

Scatter Plot: hearing(left) vs weight(kg)
weight(kg)
hearing(left)

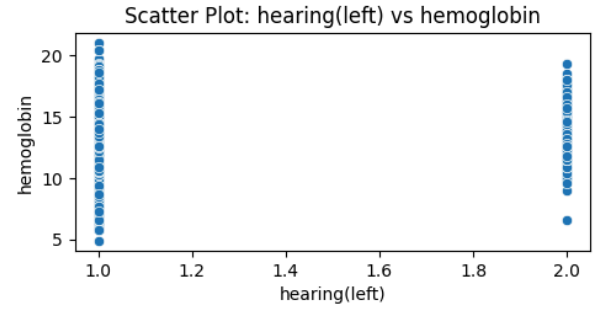

Scatter Plot: hearing(left) vs eyesight(left)
eyesight(left)
hearing(left)

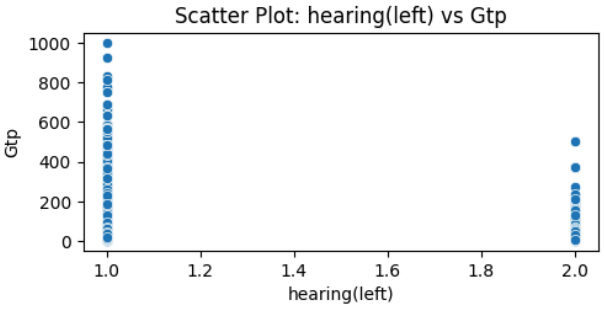

Scatter Plot: hearing(left) vs eyesight(right)
eyesight(right)
hearing(left)

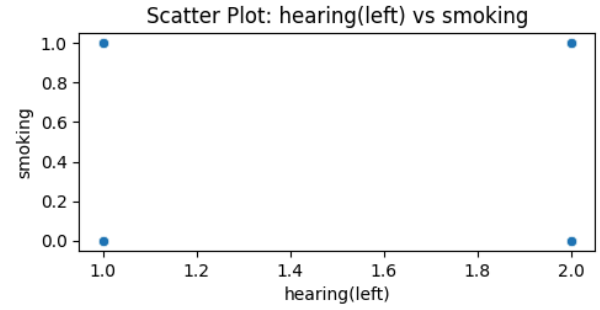

Scatter Plot: hearing(left) vs AST
AST
hearing(left)

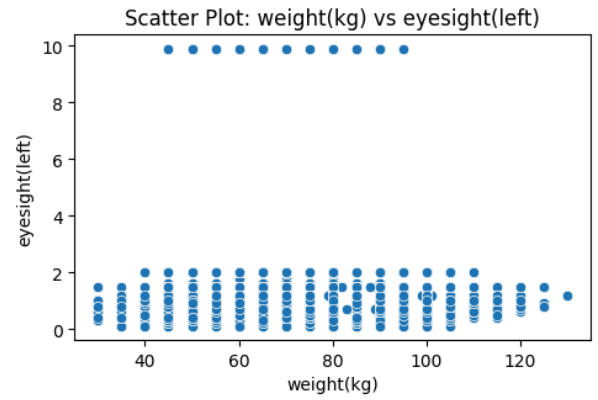

Scatter Plot: hearing(left) vs waist(cm)
waist(cm)
hearing(left)

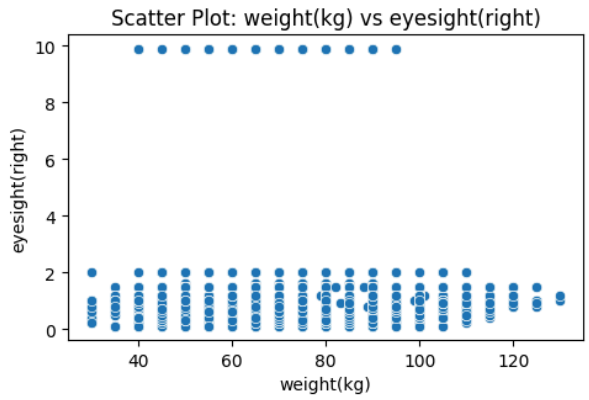

Scatter Plot: hearing(left) vs Cholesterol
Cholesterol
hearing(left)

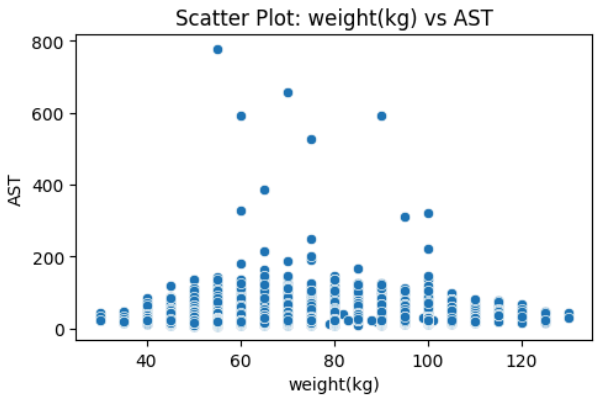

Scatter Plot: hearing(left) vs hemoglobin
hemoglobin
hearing(left)

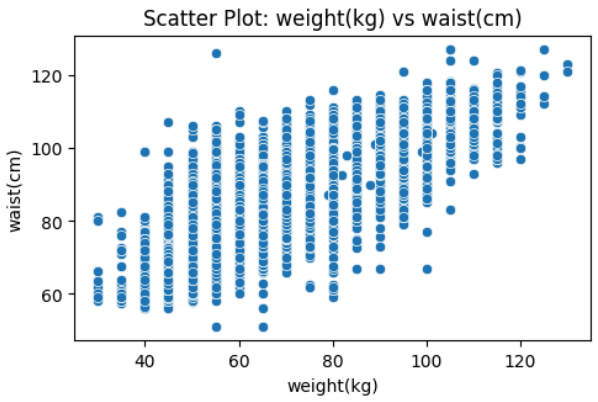

Scatter Plot: hearing(left) vs Gtp
Gtp
hearing(left)

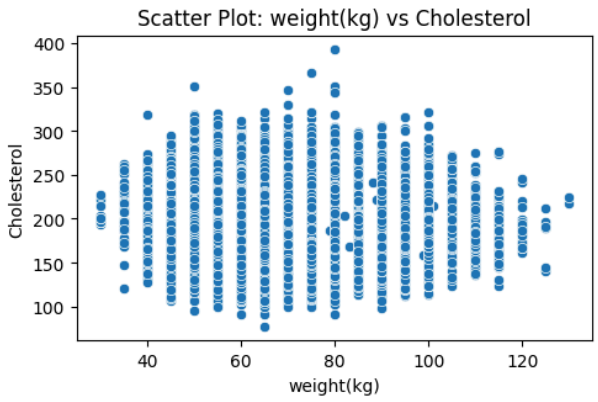

Scatter Plot: hearing(left) vs smoking
smoking
hearing(left)

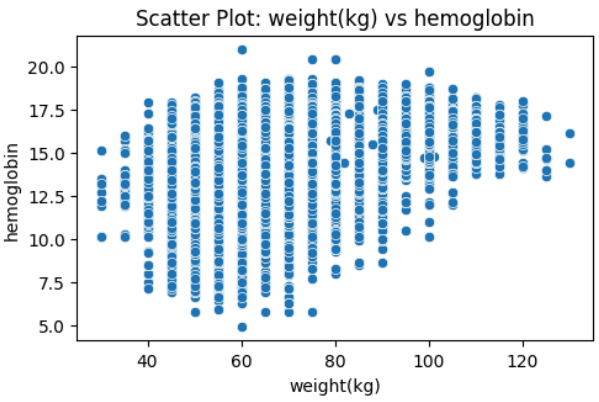

Scatter Plot: weight(kg) vs eyesight(left)
eyesight(left)
weight(kg)

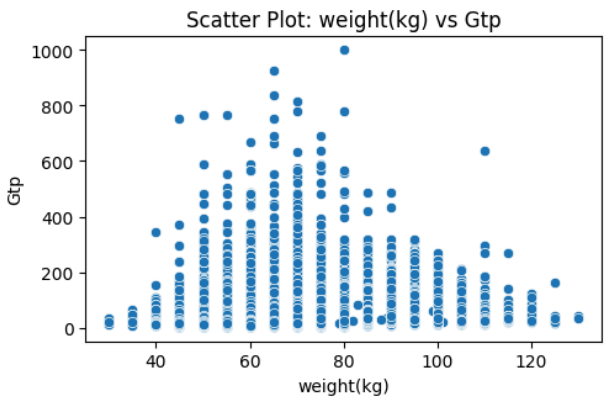

Scatter Plot: weight(kg) vs eyesight(right)
eyesight(right)
weight(kg)

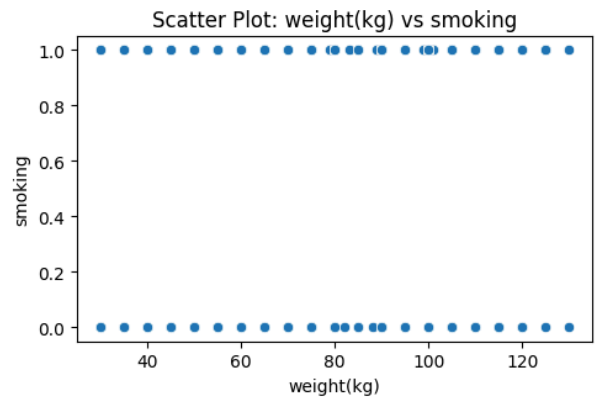

Scatter Plot: weight(kg) vs AST
AST
weight(kg)

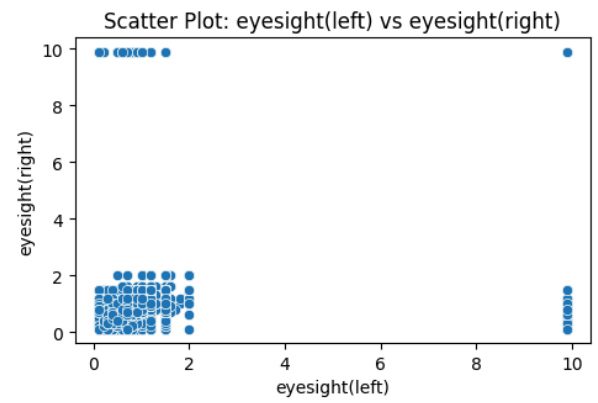

Scatter Plot: weight(kg) vs waist(cm)
waist(cm)
weight(kg)

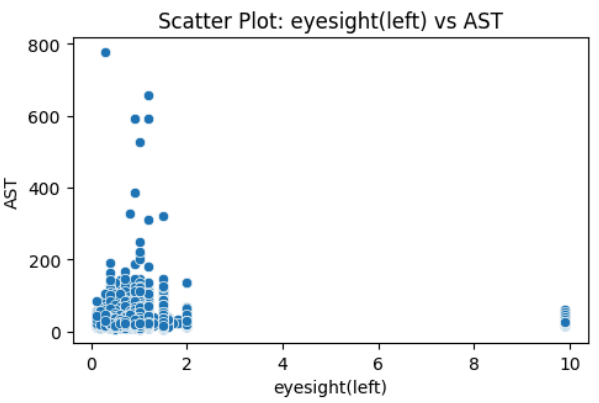

Scatter Plot: weight(kg) vs Cholesterol
Cholesterol
weight(kg)

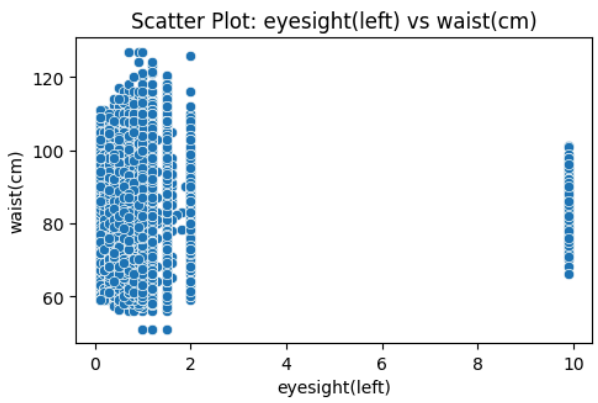

Scatter Plot: weight(kg) vs hemoglobin
hemoglobin
weight(kg)

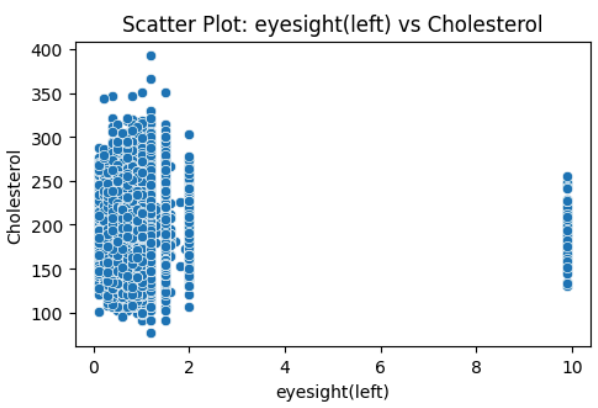

Scatter Plot: weight(kg) vs Gtp
Gtp
weight(kg)

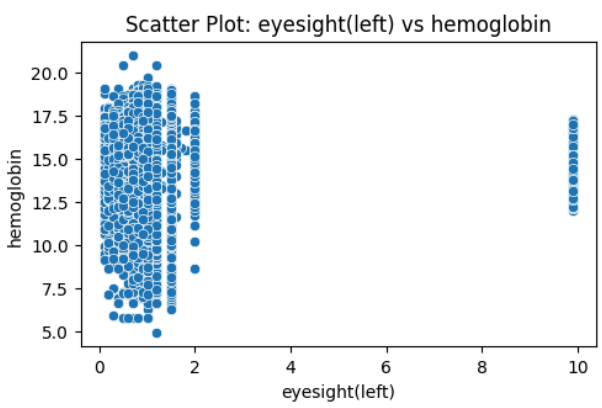

Scatter Plot: weight(kg) vs smoking
smoking
weight(kg)

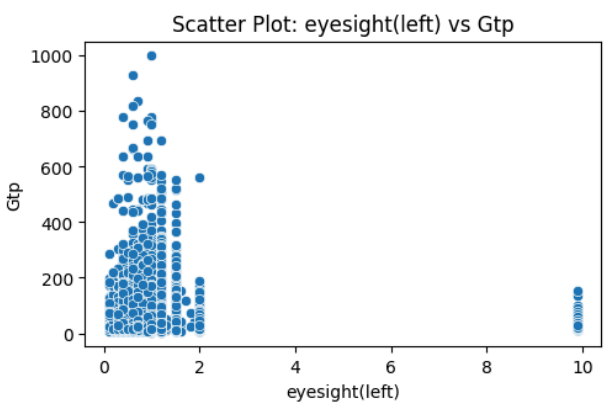

Scatter Plot: eyesight(left) vs eyesight(right)
eyesight(right)
eyesight(left)

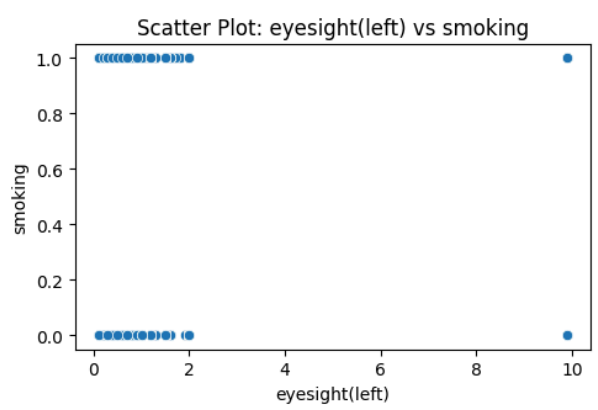

Scatter Plot: eyesight(left) vs AST
AST
eyesight(left)

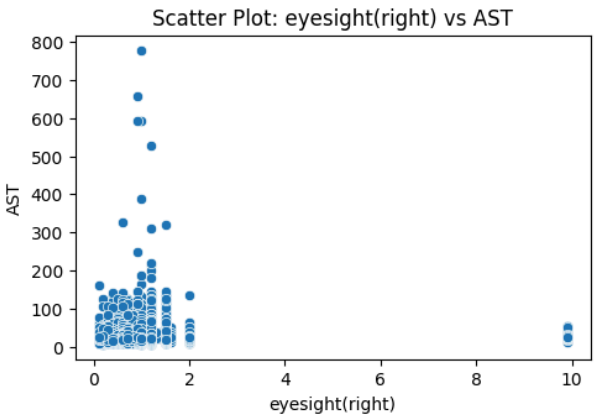

Scatter Plot: eyesight(left) vs waist(cm)
waist(cm)
eyesight(left)

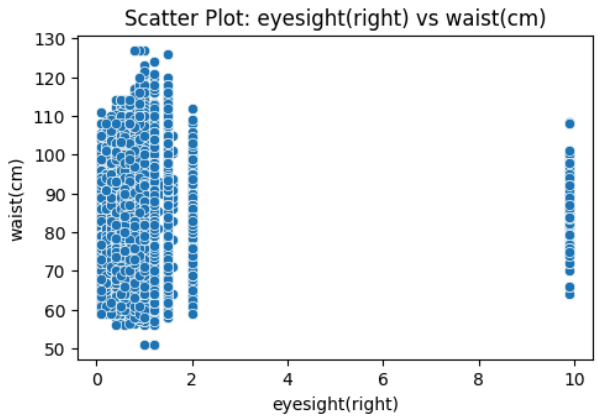

Scatter Plot: eyesight(left) vs Cholesterol
Cholesterol
eyesight(left)

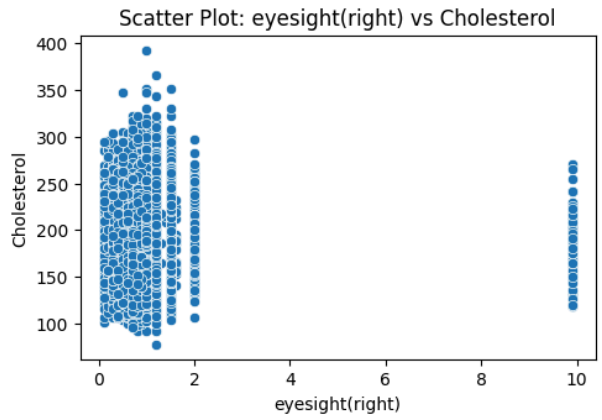

Scatter Plot: eyesight(left) vs hemoglobin
hemoglobin
eyesight(left)

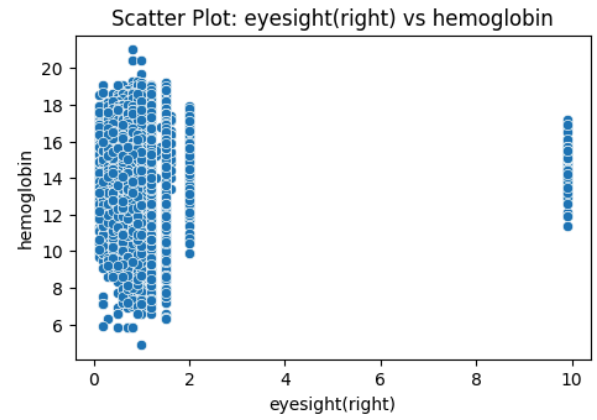

Scatter Plot: eyesight(left) vs Gtp
Gtp
eyesight(left)

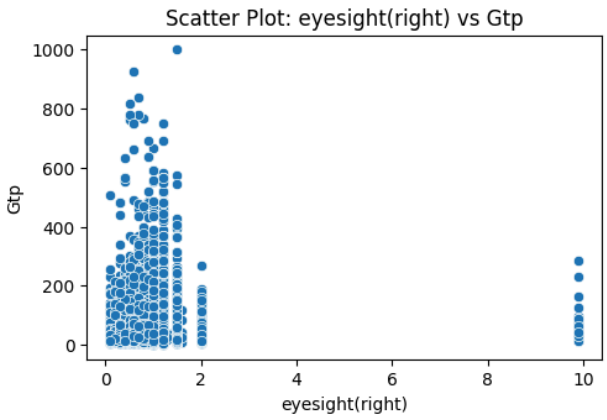

Scatter Plot: eyesight(left) vs smoking
smoking
eyesight(left)

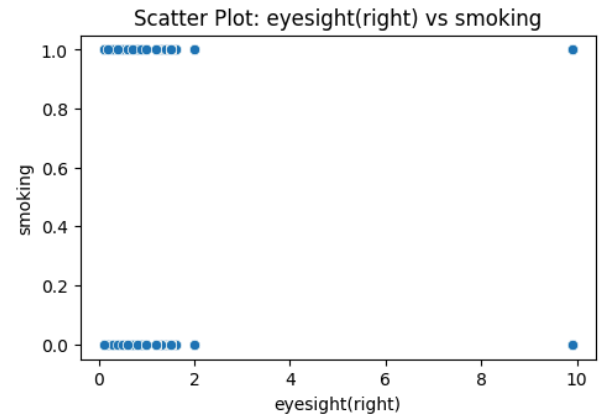

Scatter Plot: eyesight(right) vs AST
AST
eyesight(right)

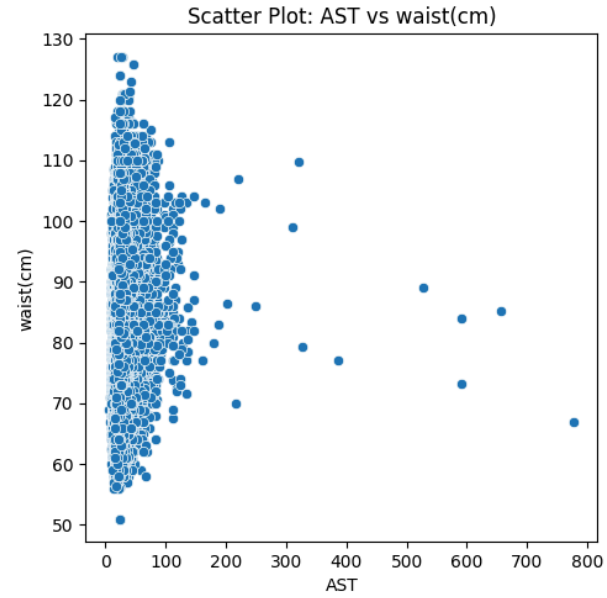

Scatter Plot: eyesight(right) vs waist(cm)
waist(cm)
eyesight(right)

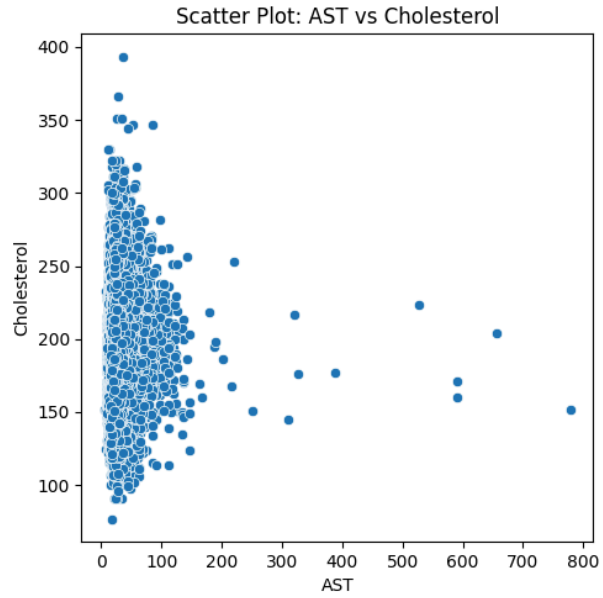

Scatter Plot: eyesight(right) vs Cholesterol
Cholesterol
eyesight(right)

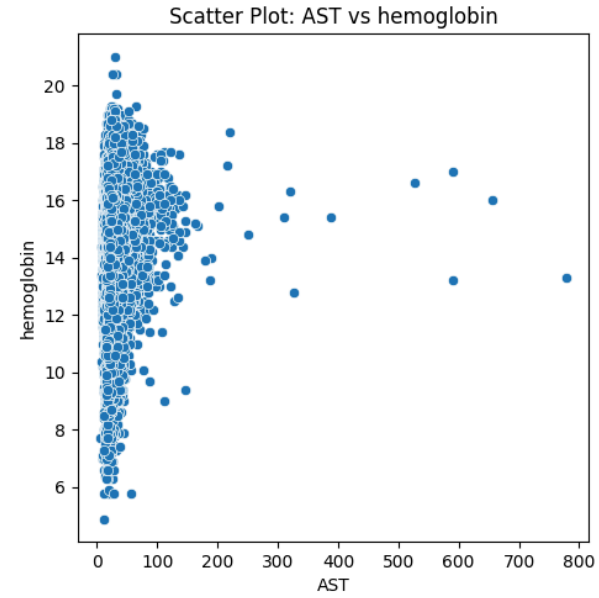

Scatter Plot: eyesight(right) vs hemoglobin
hemoglobin
eyesight(right)

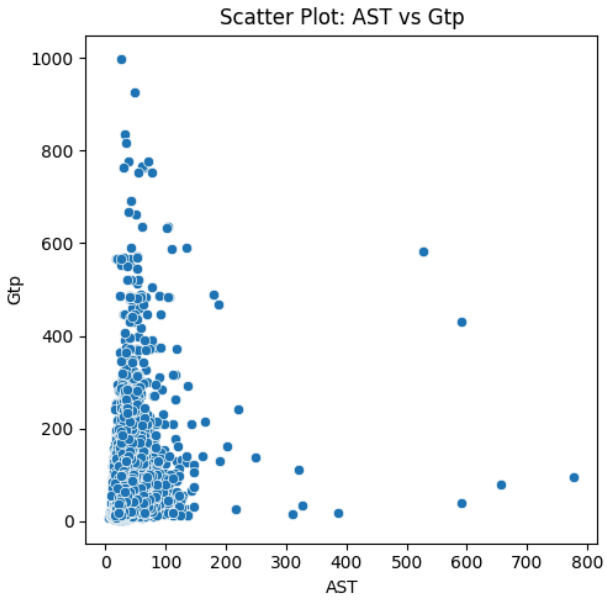

Scatter Plot: eyesight(right) vs Gtp
Gtp
eyesight(right)

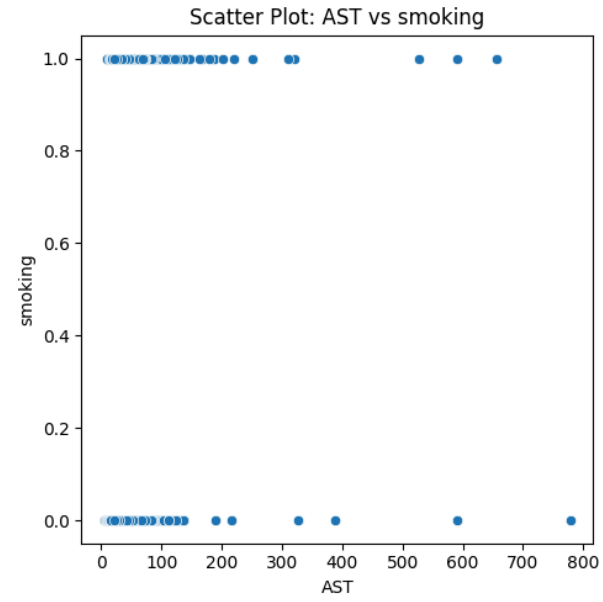

Scatter Plot: eyesight(right) vs smoking
smoking
eyesight(right)

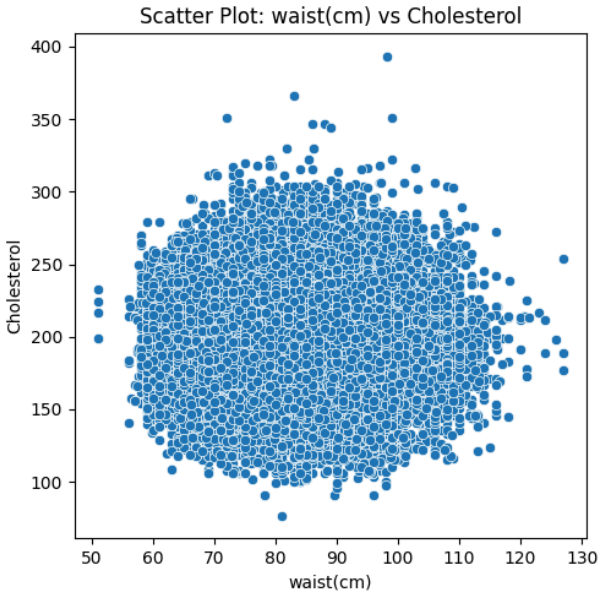

Scatter Plot: AST vs waist(cm)
waist(cm)
AST

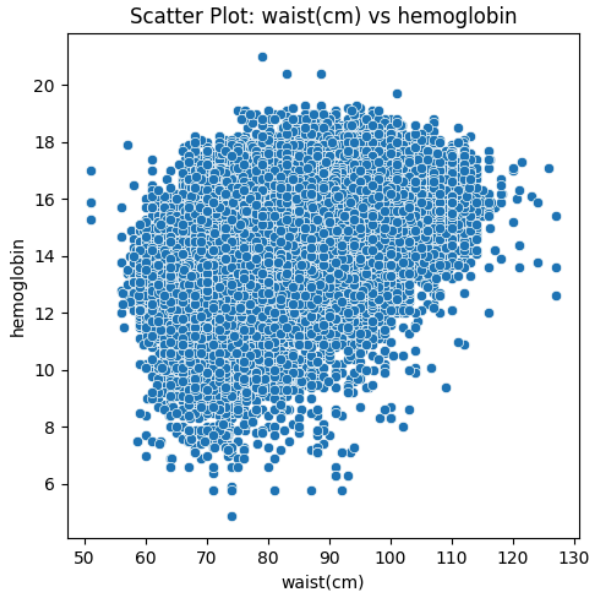

Scatter Plot: AST vs Cholesterol
Cholesterol
AST

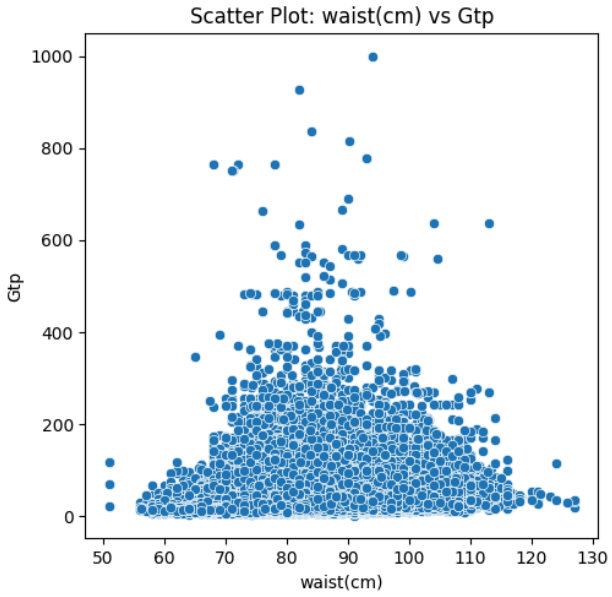

Scatter Plot: AST vs hemoglobin
hemoglobin
AST

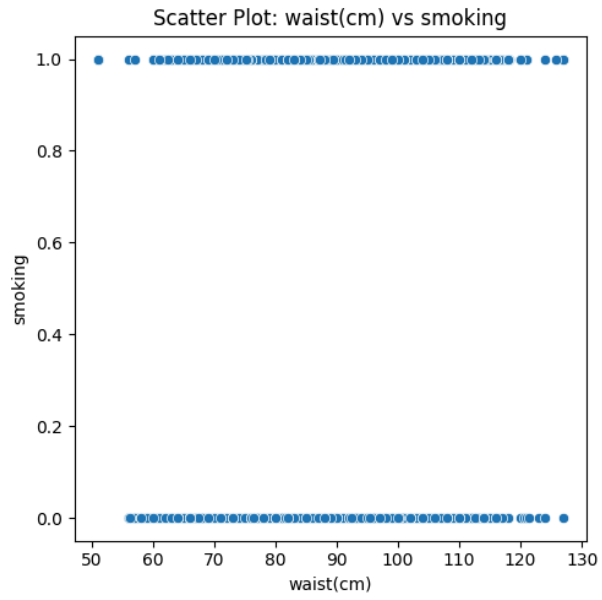

Scatter Plot: AST vs Gtp
Gtp
AST

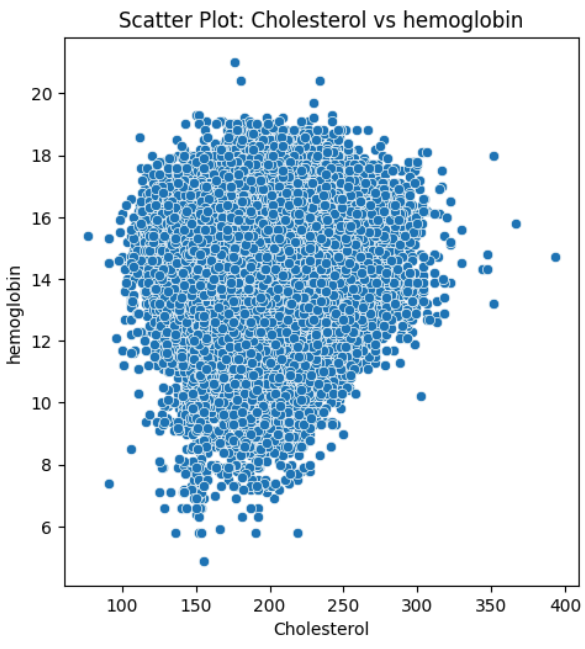

Scatter Plot: AST vs smoking
smoking
AST

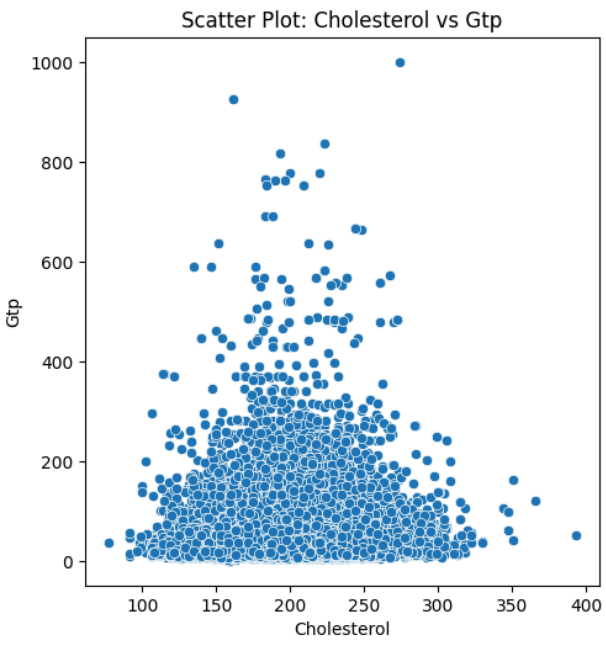

Scatter Plot: waist(cm) vs Cholesterol
Cholesterol
waist(cm)

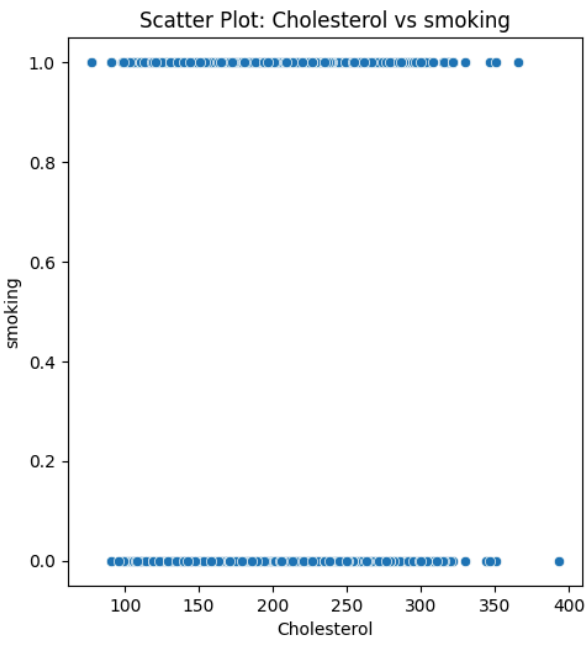

Scatter Plot: waist(cm) vs hemoglobin
hemoglobin
waist(cm)

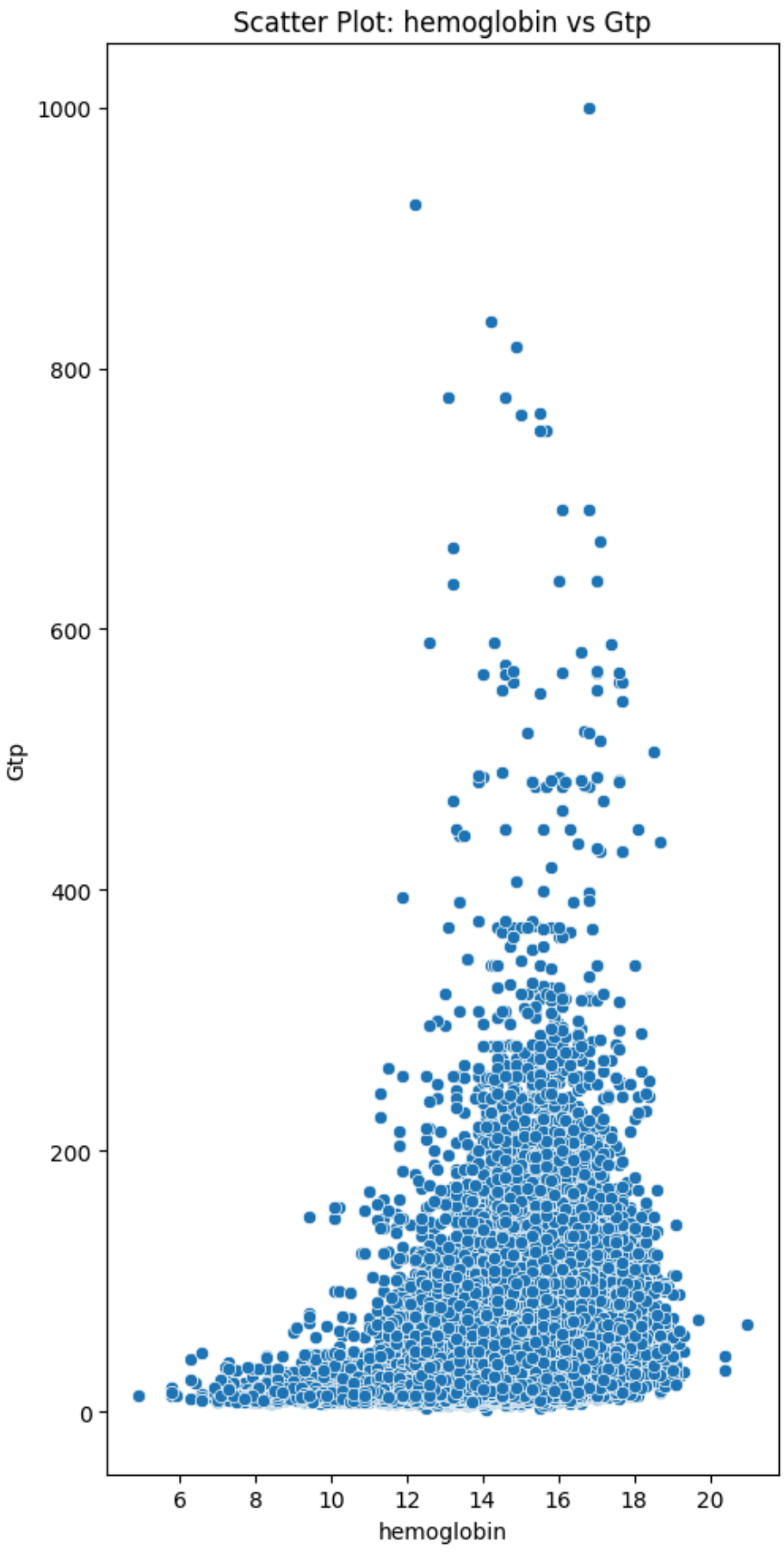

Scatter Plot: waist(cm) vs Gtp
Gtp
waist(cm)

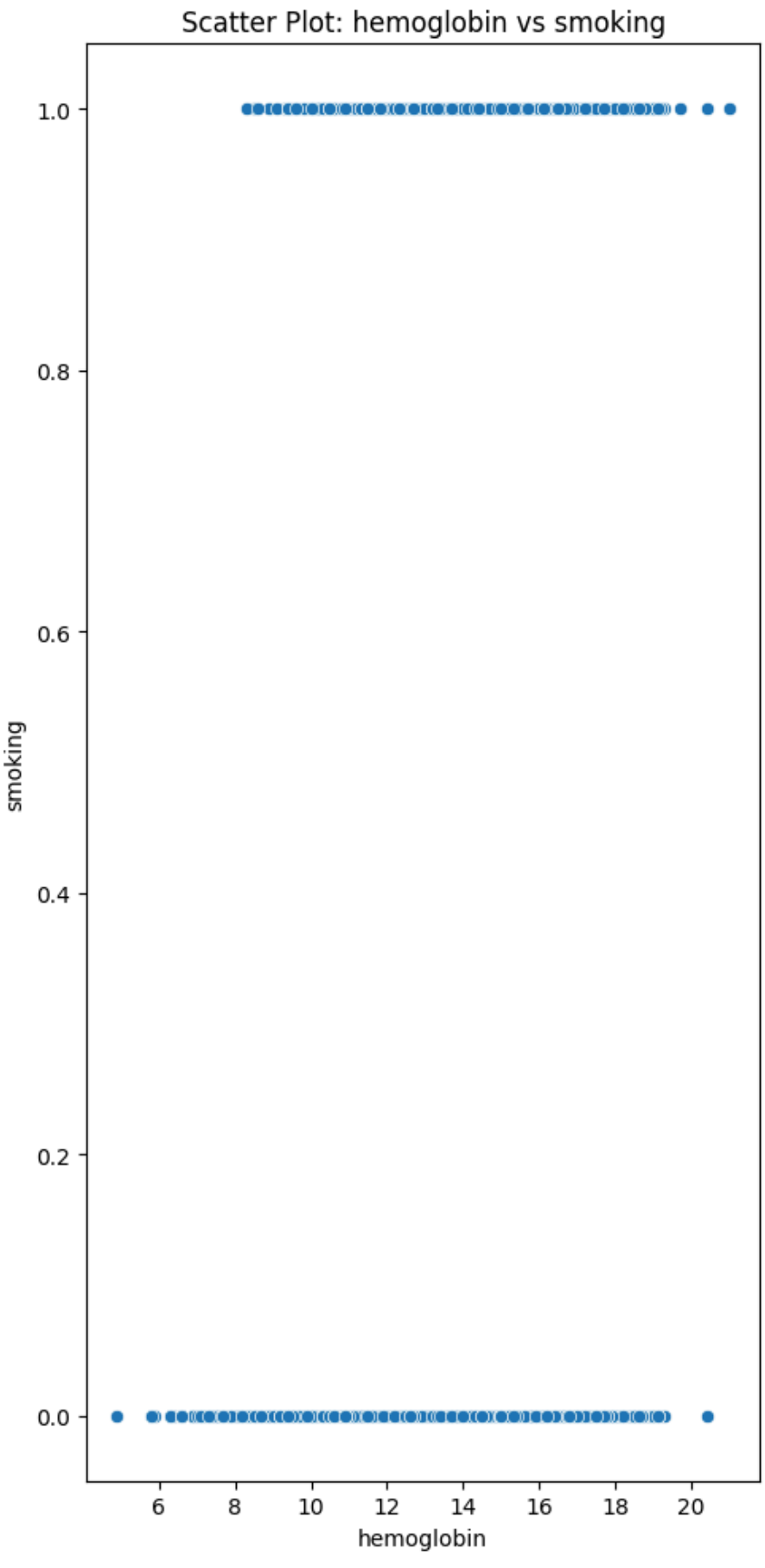

Scatter Plot: waist(cm) vs smoking
smoking
waist(cm)

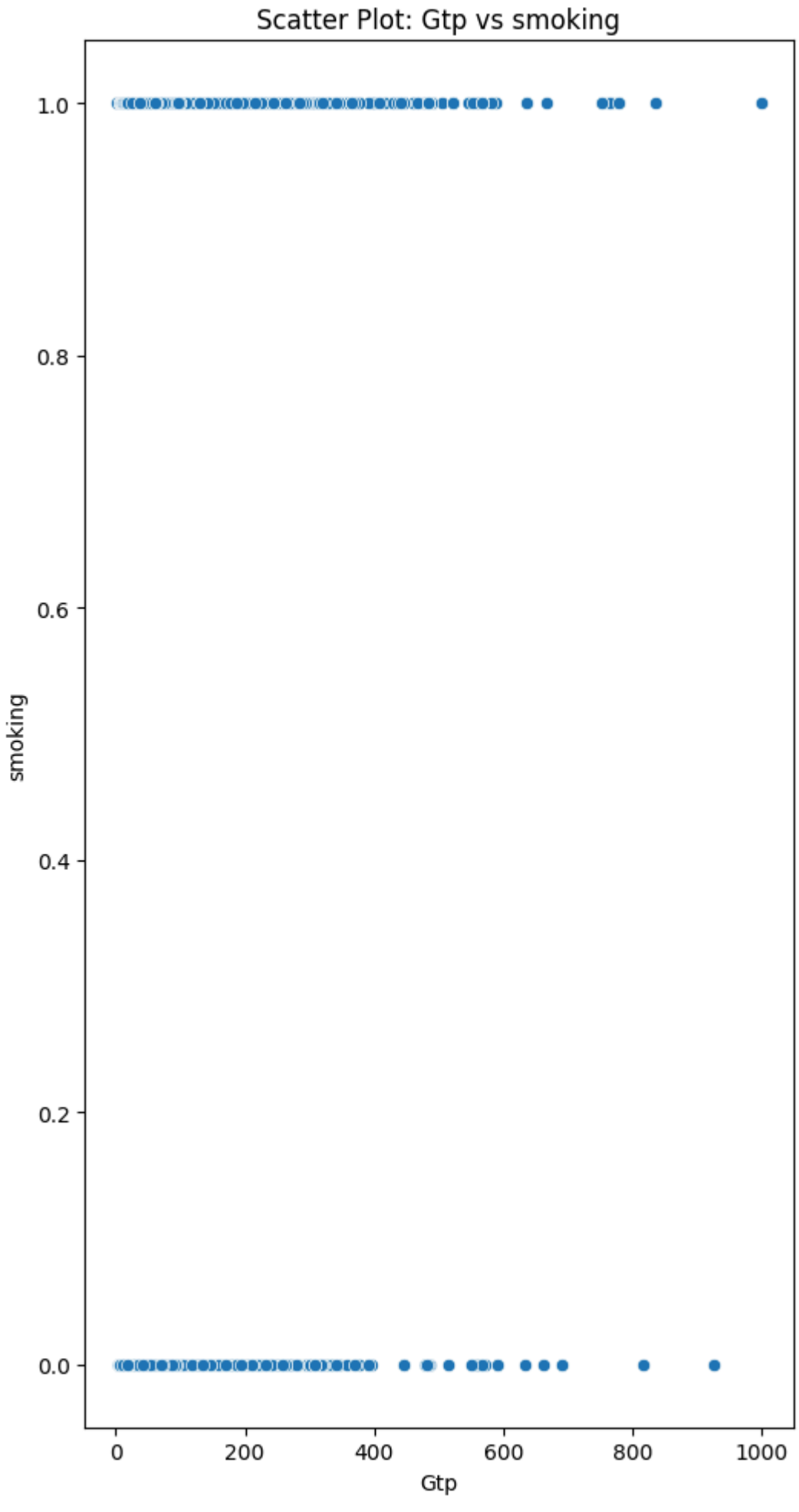

Scatter Plot: Cholesterol vs hemoglobin
hemoglobin
Cholesterol


Scatter Plot: Cholesterol vs Gtp
Gtp
Cholesterol


Scatter Plot: Cholesterol vs smoking
smoking
Cholesterol


Scatter Plot: hemoglobin vs Gtp
Gtp
hemoglobin
0
200
400
600
800
1000
6
8
10
12
14
16
18
20


Scatter Plot: hemoglobin vs smoking
smoking
hemoglobin
0.0
0.2
0.4
0.6
0.8
1.0
6
8
10
12
14
16
18
20

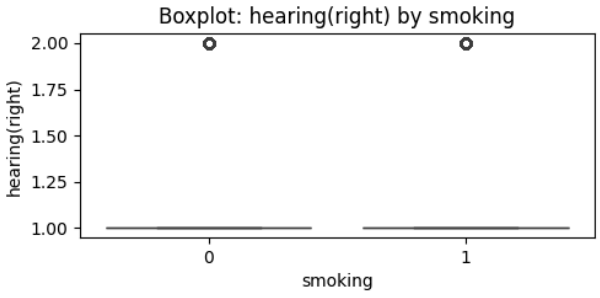


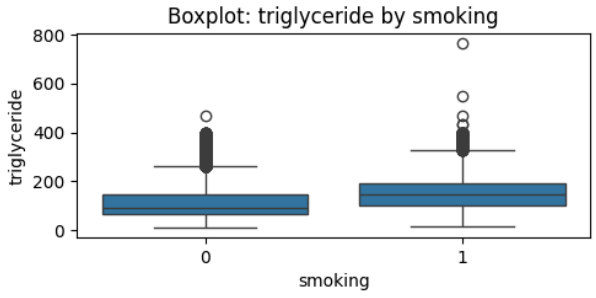
scatter plots of all the data colums against each other

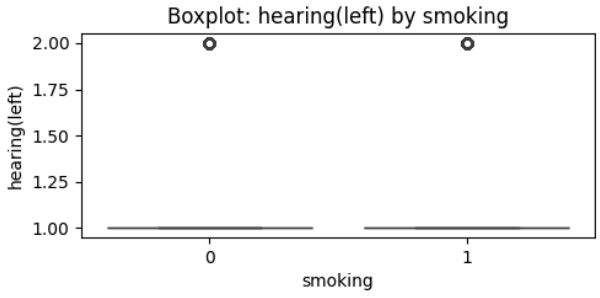
Boxplot

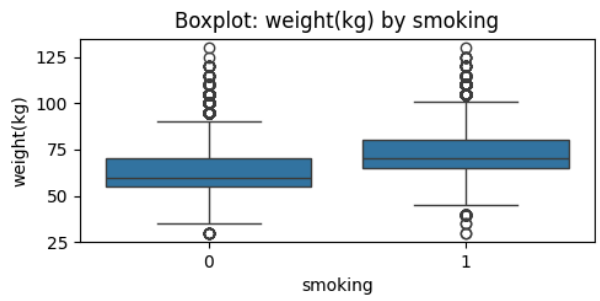
In [23]:
```
categorical_columns = ['smoking', 'hearing(right)', 'hearing(left)']
numerical_columns = df.select_dtypes(include=['float64', 'int64']).columns

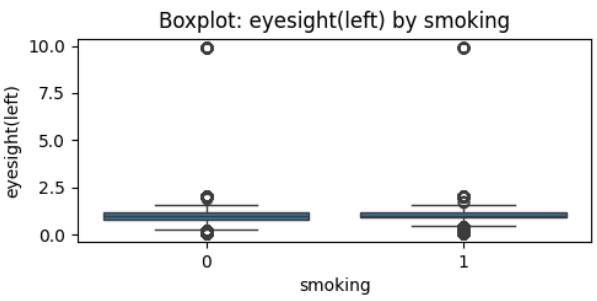
for cat_col in categorical_columns:
    plt.figure(figsize=(15, 10))
    num_plots = len(numerical_columns) - 1
    rows = (num_plots // 3) + 1
    for i, num_col in enumerate(numerical_columns):
        if num_col != cat_col:
            plt.subplot(rows, 3, i + 1)
            sns.boxplot(x=df[cat_col], y=df[num_col])
            plt.title(f'Boxplot: {num_col} by {cat_col}')
            plt.xlabel(cat_col)
            plt.ylabel(num_col)
    plt.tight_layout()
    plt.show()
```

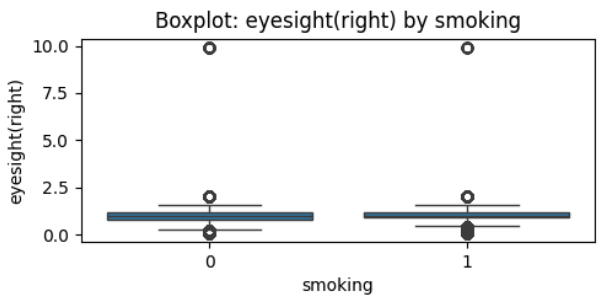

Boxplot: hearing(right) by smoking
hearing(right)
smoking

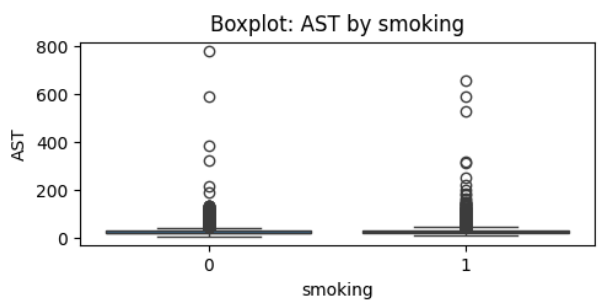

Boxplot: triglyceride by smoking
triglyceride
smoking


Boxplot: hearing(left) by smoking
hearing(left)
smoking

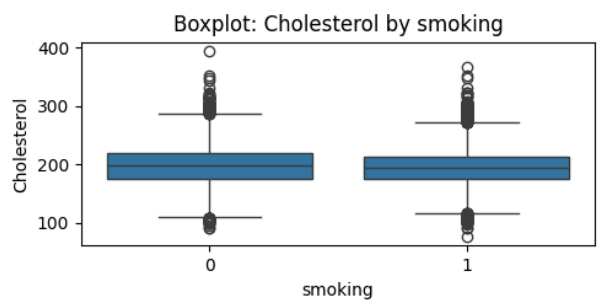

Boxplot: weight(kg) by smoking
weight(kg)
smoking

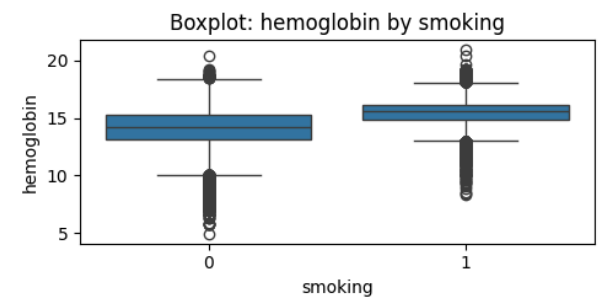

Boxplot: eyesight(left) by smoking
eyesight(left)
smoking

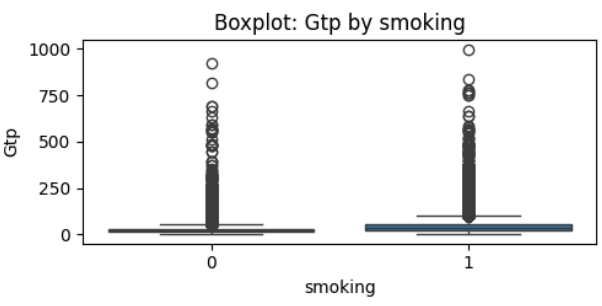

Boxplot: eyesight(right) by smoking
eyesight(right)
smoking

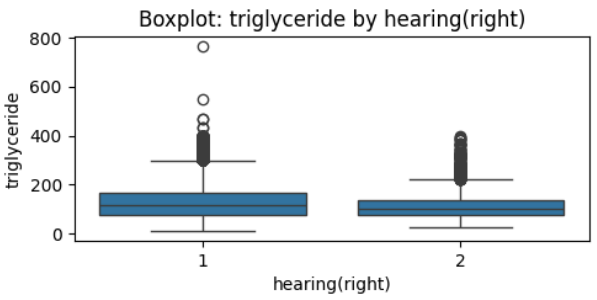

Boxplot: AST by smoking
AST
smoking

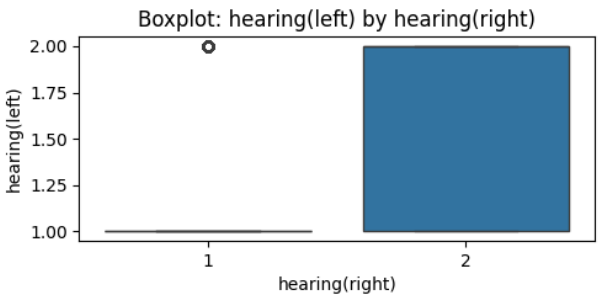

Boxplot: waist(cm) by smoking
waist(cm)
smoking

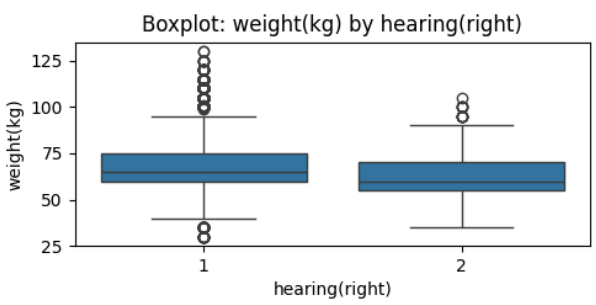

Boxplot: Cholesterol by smoking
Cholesterol
smoking

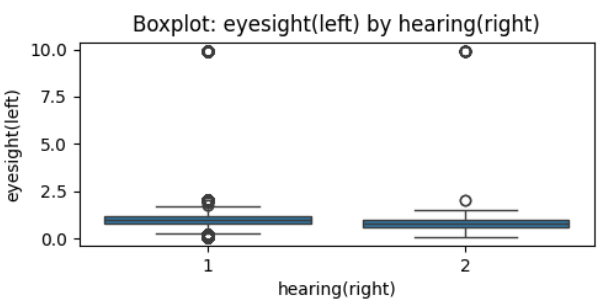

Boxplot: hemoglobin by smoking
hemoglobin
smoking

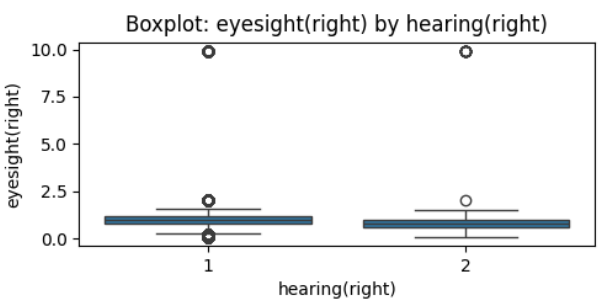

Boxplot: Gtp by smoking
Gtp
smoking

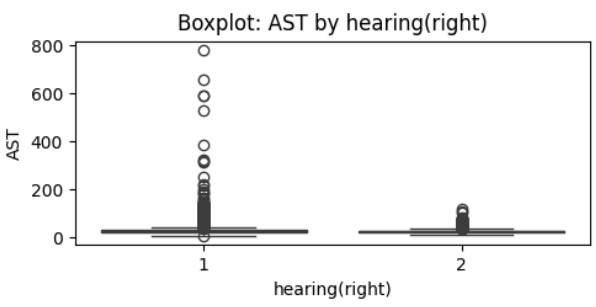

Boxplot: triglyceride by hearing(right)
triglyceride
hearing(right)

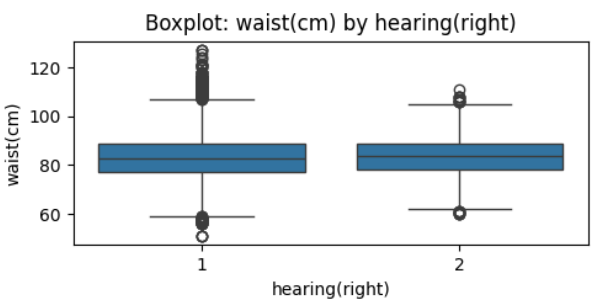

Boxplot: hearing(left) by hearing(right)
hearing(left)
hearing(right)

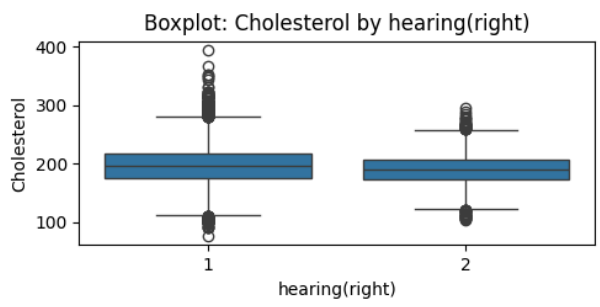

Boxplot: weight(kg) by hearing(right)
weight(kg)
hearing(right)

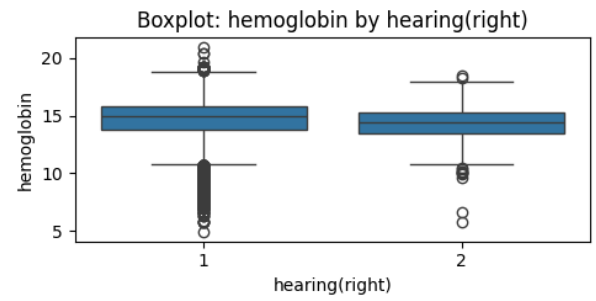

Boxplot: eyesight(left) by hearing(right)
eyesight(left)
hearing(right)

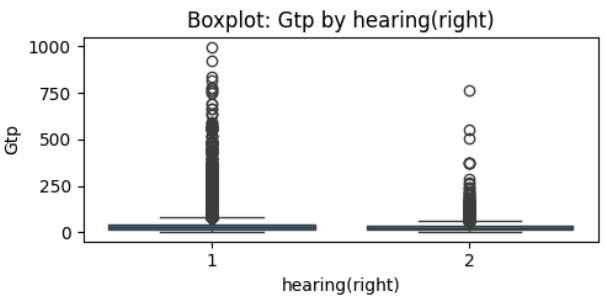

Boxplot: eyesight(right) by hearing(right)
eyesight(right)
hearing(right)

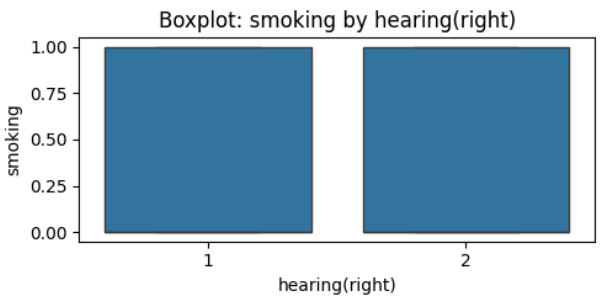

Boxplot: AST by hearing(right)
AST
hearing(right)

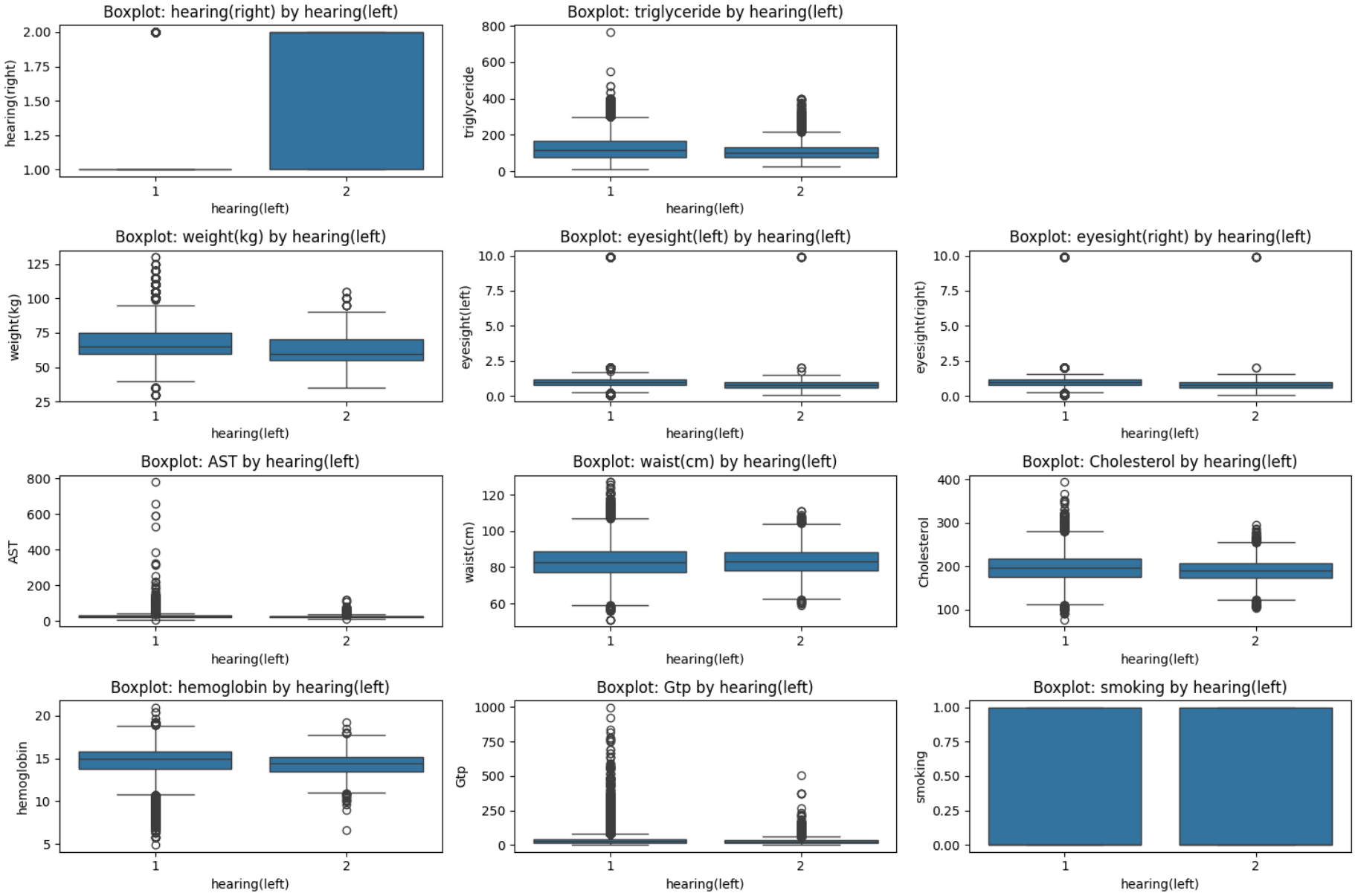

Boxplot: waist(cm) by hearing(right)
waist(cm)
hearing(right)


Boxplot: Cholesterol by hearing(right)
Cholesterol
hearing(right)


Boxplot: hemoglobin by hearing(right)
hemoglobin
hearing(right)


Boxplot: Gtp by hearing(right)
Gtp
hearing(right)


Boxplot: smoking by hearing(right)
smoking
hearing(right)

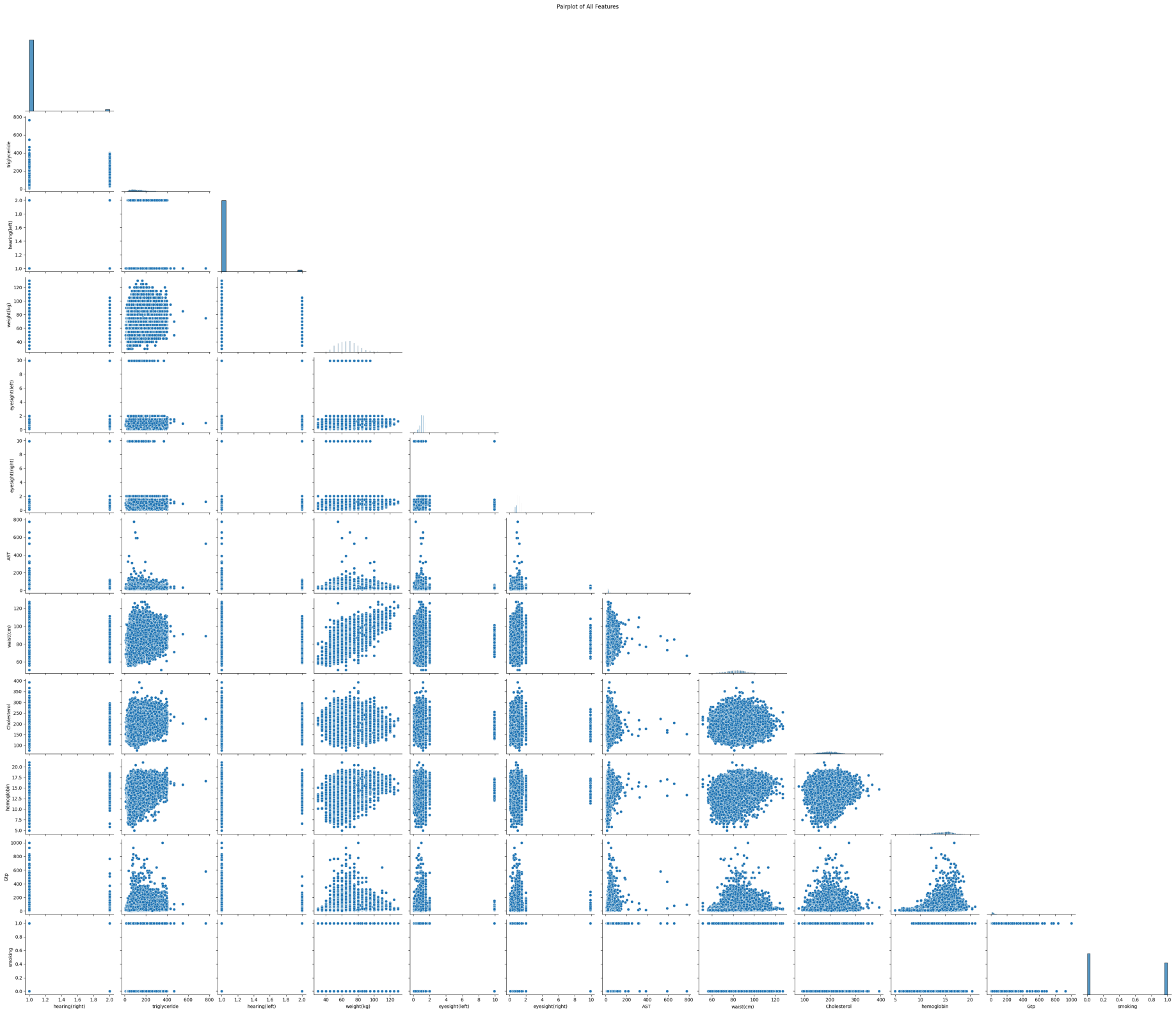


box plots of all numerical features vs (categorical features & target)

Multivariate

pairplot

In [30]:
```
g = sns.pairplot(df, diag_kind='auto', height=2.5, aspect=1.2, corner=True)
g.fig.suptitle('Pairplot of All Features', y=1.02)
plt.show()
```



3d scatter

In [25]:
```
# Weight vs Waist vs Cholesterol
fig1 = px.scatter_3d(df,
                     x='weight(kg)',
                     y='waist(cm)',
                     z='Cholesterol',
                     color='smoking',
                     title='3D Scatter Plot: Weight vs Waist vs Cholesterol')
fig1.show()

# Triglyceride vs AST vs Gtp
fig2 = px.scatter_3d(df,
                     x='triglyceride',
                     y='AST',
                     z='Gtp',
                     color='smoking',
                     title='3D Scatter Plot: Triglyceride vs AST vs Gtp')
fig2.show()

# Eyesight (left) vs Eyesight (right) vs Hemoglobin
fig3 = px.scatter_3d(df,
                     x='eyesight(left)',
                     y='eyesight(right)',
                     z='hemoglobin',
                     color='smoking',
                     title='3D Scatter Plot: Eyesight (left) vs Eyesight (right) vs Hemoglobin')
fig3.show()
```

dimensionality reduction (PCA)

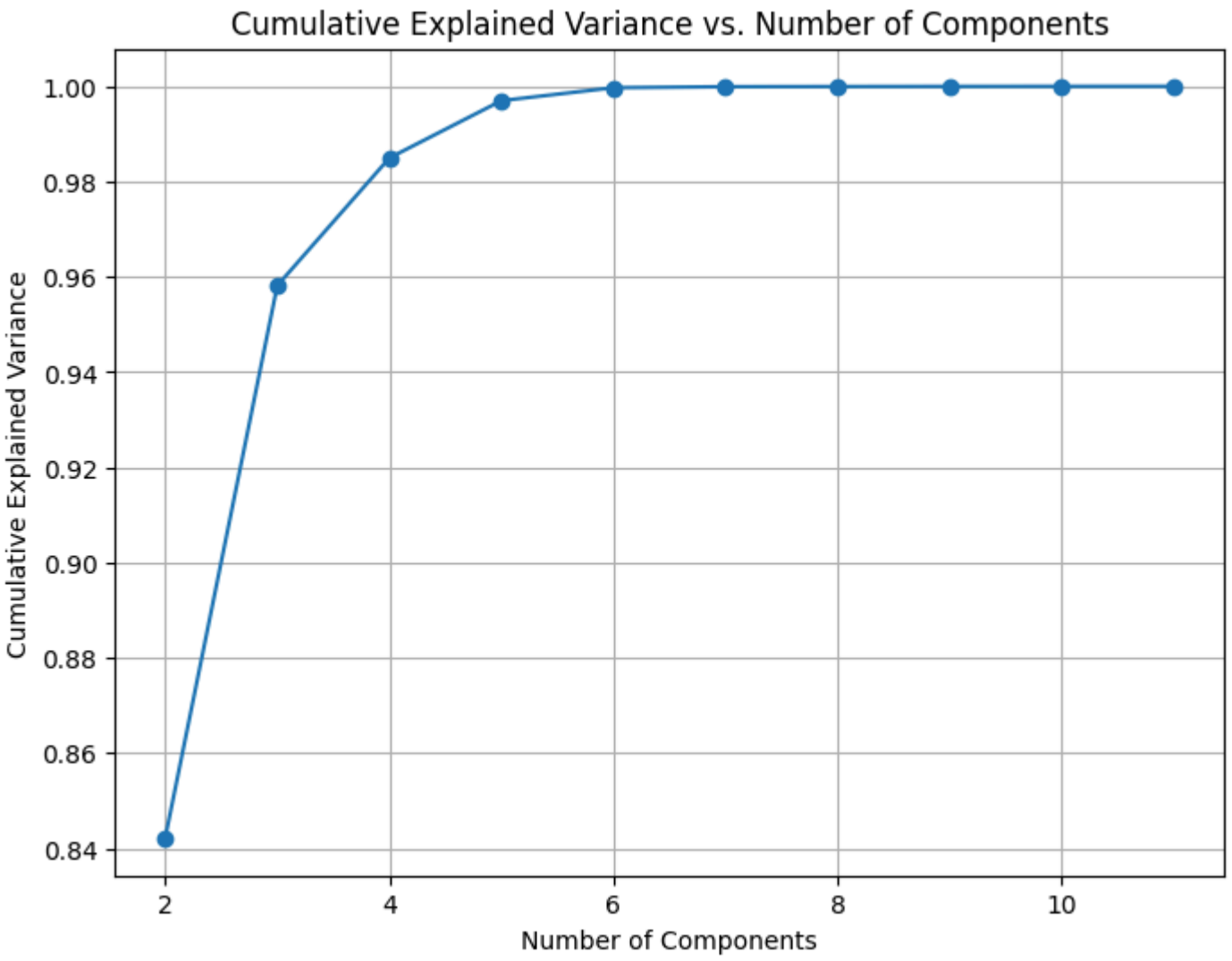
In [31]:
```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import numpy as np

components_range = range(2, 12)

cumulative_variance = []
```

```
for n_components in components_range:
    pca = PCA(n_components=n_components)
    X_pca = pca.fit_transform(df)
    cumulative_variance.append(np.sum(pca.explained_variance_ratio_))

plt.figure(figsize=(8, 6))
plt.plot(components_range, cumulative_variance, marker='o')
plt.title('Cumulative Explained Variance vs. Number of Components')
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
plt.grid(True)
plt.show()
```

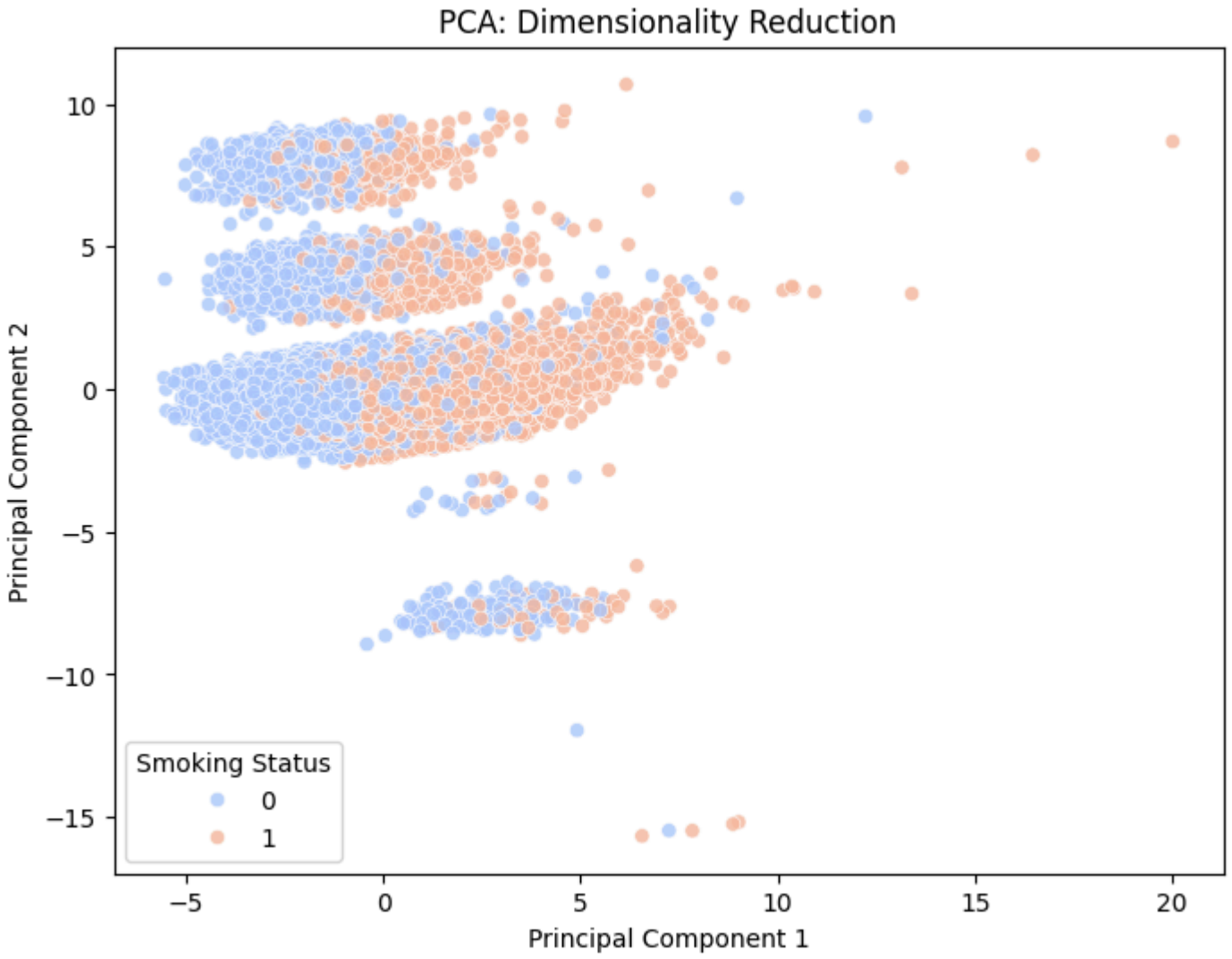


Graph to determine principle components in Dim. reduction

In [32]:
```
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df.select_dtypes(include=["float64", "int64"]))

pca = PCA(n_components=2)
pca_data = pca.fit_transform(scaled_data)
pca_df = pd.DataFrame(data=pca_data, columns=["PC1", "PC2"])
pca_df["smoking"] = df["smoking"]

plt.figure(figsize=(8, 6))
sns.scatterplot(data=pca_df, x="PC1", y="PC2", hue="smoking", palette="coolwarm", alpha=0.8)
plt.title("PCA: Dimensionality Reduction")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend(title="Smoking Status")
plt.show()
```



It provides a representation of the dataset making it easier to visualize and analyze multivariate data

Parallel Coordinates Plot

In [27]:
```
plt.figure(figsize=(15, 8))
parallel_coordinates(df, 'smoking', colormap='viridis')
plt.title('Parallel Coordinates Plot')
plt.xlabel('Features')
plt.ylabel('Values')
plt.legend(title='Smoking Status', loc='upper left')
plt.show()
```

